

项目2 数据可视化



数据可视化指的是通过可视化表示来探索数据,它与**数据挖掘**紧密相关,而**数据挖掘**指的是使用代码来探索数据集的规律和关联。数据集可以用一行代码就能表示的小型数字列表,也可以是数以吉字节的数据。

漂亮地呈现数据关乎的并非仅仅是漂亮的图片。以引人注目的简洁方式呈现数据,让观看者能够明白其含义,发现数据集中原本未意识到的规律和意义。

所幸即便没有超级计算机,也能够可视化复杂的数据。鉴于Python的高效性,使用它在笔记本电脑上就能快速地探索由数百万个数据点组成的数据集。数据点并非必须是数字,利用本书前半部分介绍的基本知识,也可以对非数字数据进行分析。

在基因研究、天气研究、政治经济分析等众多领域,大家都使用Python来完成数据密集型工作。数据科学家使用Python编写了一系列令人印象深刻的可视化和分析工具,其中很多也可供你使用。最流行的工具之一是matplotlib,它是一个数学绘图库,我们将使用它来制作简单的图表,如折线图 and 散点图。然后,我们将基于随机漫步概念生成一个更有趣的数据集——根据一系列随机决策生成的图表。

我们还将使用Pygal包,它专注于生成适合在数字设备上显示的图表。通过使用Pygal,可在用户与图表交互时突出元素以及调整其大小,还可轻松地调整整个图表的尺寸,使其适合在微型智能手表或巨型显示器上显示。我们将使用Pygal以各种方式探索掷骰子的结果。

15.1 安装 matplotlib

首先,需要安装matplotlib,我们将使用它来制作开始的几个图表。如果你还未使用过pip,请参阅12.2.1节。

15.1.1 在 Linux 系统中安装 matplotlib

如果你使用的是系统自带的Python版本，可使用系统的包管理器来安装matplotlib，为此只需执行一行命令：

```
$ sudo apt-get install python3-matplotlib
```

如果你使用的是Python 2.7，请执行如下命令：

```
$ sudo apt-get install python-matplotlib
```

如果你安装了较新的Python版本，就必须安装matplotlib依赖的一些库：

```
$ sudo apt-get install python3.5-dev python3.5-tk tk-dev  
$ sudo apt-get install libfreetype6-dev g++
```

再使用pip来安装matplotlib：

```
$ pip install --user matplotlib
```

15.1.2 在 OS X 系统中安装 matplotlib

Apple的标准Python安装自带了matplotlib。要检查系统是否安装了matplotlib，可打开一个终端会话并尝试导入matplotlib。如果系统没有自带matplotlib，且你的Python是使用Homebrew安装的，则可以像下面这样安装matplotlib：

```
$ pip install --user matplotlib
```

注意 安装包时可能需要使用pip3，而不是pip。另外，如果这个命令不管用，你可能需要删除标志--user。

15.1.3 在 Windows 系统中安装 matplotlib

在Windows系统中，首先需要安装Visual Studio。为此，请访问<https://dev.windows.com/>，单击Downloads，再查找Visual Studio Community——一组免费的Windows开发工具。请下载并运行该安装程序。

接下来，需要下载matplotlib安装程序。为此，请访问<https://pypi.python.org/pypi/matplotlib/>，并查找与你使用的Python版本匹配的wheel文件（扩展名为.whl的文件）。例如，如果你使用的是32位的Python 3.5，则需要下载matplotlib-1.4.3-cp35-none-win32.whl。

注意 如果找不到与你安装的Python版本匹配的文件，请去<http://www.lfd.uci.edu/~gohlke/pythonlibs/#matplotlib>看看，这个网站发布安装程序的时间通常比matplotlib官网早些。

将这个.whl文件复制到你的项目文件夹，打开一个命令窗口，并切换到该项目文件夹，再使用pip来安装matplotlib：

```
> cd python_work
python_work> python -m pip install --user matplotlib-1.4.3-cp35-none-win32.whl
```

15.1.4 测试 matplotlib

安装必要的包后，对安装进行测试。为此，首先使用命令python或python3启动一个终端会话，再尝试导入matplotlib：

```
$ python3
>>> import matplotlib
>>>
```

如果没有出现任何错误消息，就说明你的系统安装了matplotlib，可以接着阅读下一节。

注意 如果你在安装过程中遇到了麻烦，请参阅附录C。如果依然无济于事，请向他人寻求帮助。对于你遇到的问题，只要向经验丰富的Python程序员提供少量的信息，他们很可能很快就能帮你解决。

15.1.5 matplotlib 画廊

要查看使用matplotlib可制作的各种图表，请访问<http://matplotlib.org/>的示例画廊。单击画廊中的图表，就可查看用于生成图表的代码。

15.2 绘制简单的折线图

下面来使用matplotlib绘制一个简单的折线图，再对其进行定制，以实现信息更丰富的数据可视化。我们将使用平方数序列1、4、9、16和25来绘制这个图表。

只需向matplotlib提供如下数字，matplotlib就能完成其他的工作：

```
mpl_squares.py

import matplotlib.pyplot as plt

squares = [1, 4, 9, 16, 25]
```

```
plt.plot(squares)
plt.show()
```

我们首先导入了模块pyplot，并给它指定了别名plt，以免反复输入pyplot。在线示例大都这样做，因此这里也这样做。模块pyplot包含很多用于生成图表的函数。

我们创建了一个列表，在其中存储了前述平方数，再将这个列表传递给函数plot()，这个函数尝试根据这些数字绘制出有意义的图形。plt.show()打开matplotlib查看器，并显示绘制的图形，如图15-1所示。查看器让你能够缩放和导航图形，另外，单击磁盘图标可将图形保存起来。

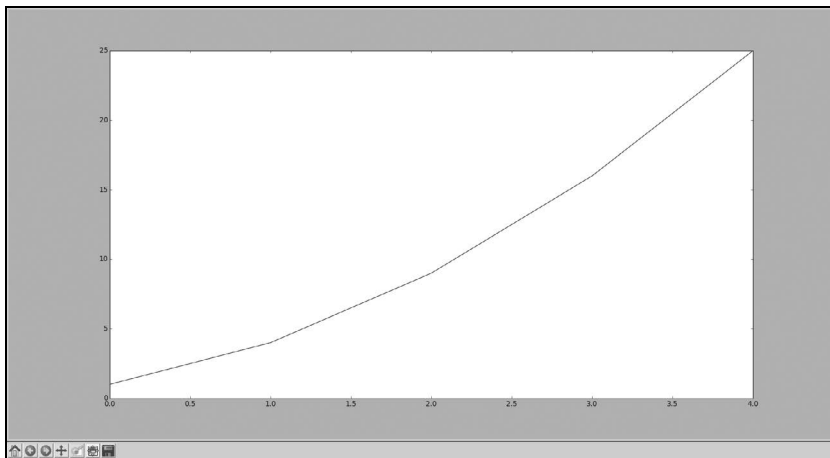


图15-1 使用matplotlib可制作的最简单的图表

15.2.1 修改标签文字和线条粗细

图15-1所示的图形表明数字是越来越大的，但标签文字太小，线条太细。所幸matplotlib让你能够调整可视化的各个方面。

下面通过一些定制来改善这个图形的可读性，如下所示：

mpl_squares.py

```
import matplotlib.pyplot as plt

squares = [1, 4, 9, 16, 25]
❶ plt.plot(squares, linewidth=5)

# 设置图表标题，并给坐标轴加上标签
❷ plt.title("Square Numbers", fontsize=24)
❸ plt.xlabel("Value", fontsize=14)
   plt.ylabel("Square of Value", fontsize=14)

# 设置刻度标记的大小
❹ plt.tick_params(axis='both', labels=14)
```

```
plt.show()
```

参数`linewidth`（见❶）决定了`plot()`绘制的线条的粗细。函数`title()`（见❷）给图表指定标题。在上述代码中，出现了多次的参数`fontsize`指定了图表中文字的大小。

函数`xlabel()`和`ylabel()`让你能够为每条轴设置标题（见❸）；而函数`tick_params()`设置刻度的样式（见❹），其中指定的实参将影响x轴和y轴上的刻度（`axes='both'`），并将刻度标记的字号设置为14（`labelsize=14`）。

最终的图表阅读起来容易得多了，如图15-2所示：标签文字更大，线条也更粗。

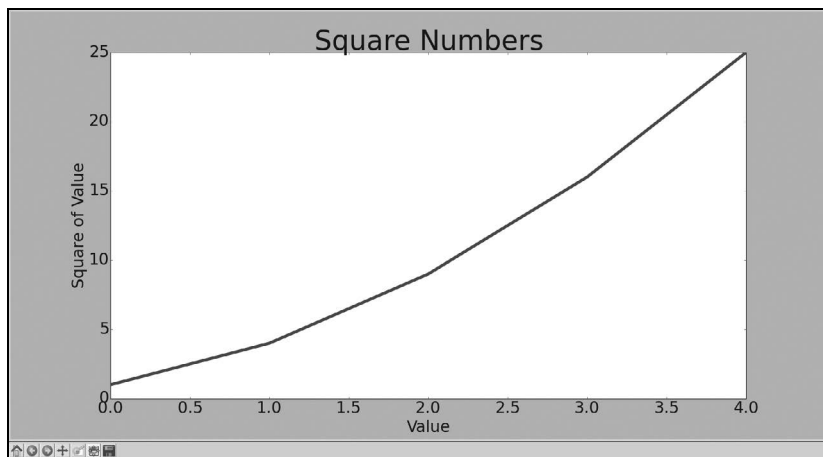


图15-2 现在图表阅读起来容易得多

15.2.2 校正图形

图形更容易阅读后，我们发现没有正确地绘制数据：折线图的终点指出4.0的平方为25！下面来修复这个问题。

当你向`plot()`提供一系列数字时，它假设第一个数据点对应的x坐标值为0，但我们的第一个点对应的x值为1。为改变这种默认行为，我们可以给`plot()`同时提供输入值和输出值：

`mpl_squares.py`

```
import matplotlib.pyplot as plt

input_values = [1, 2, 3, 4, 5]
squares = [1, 4, 9, 16, 25]
plt.plot(input_values, squares, linewidth=5)
```

```
# 设置图表标题并给坐标轴加上标签
--snip--
```

现在`plot()`将正确地绘制数据，因为我们同时提供了输入值和输出值，它无需对输出值的生成方式作出假设。最终的图形是正确的，如图15-3所示。

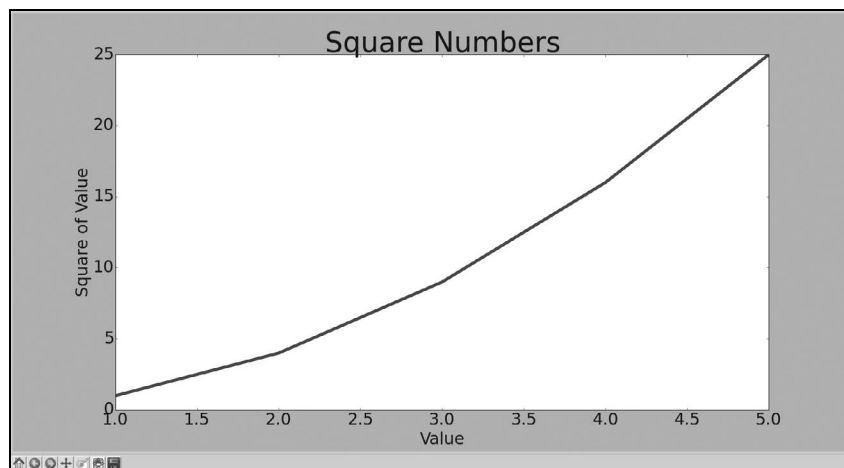


图15-3 根据数据正确地绘制了图形

使用`plot()`时可指定各种实参，还可使用众多函数对图形进行定制。本章后面处理更有趣的数据集时，将继续探索这些定制函数。

15.2.3 使用 `scatter()` 绘制散点图并设置其样式

有时候，需要绘制散点图并设置各个数据点的样式。例如，你可能想以一种颜色显示较小的值，而用另一种颜色显示较大的值。绘制大型数据集时，你还可以对每个点都设置同样的样式，再使用不同的样式选项重新绘制某些点，以突出它们。

要绘制单个点，可使用函数`scatter()`，并向它传递一对`x`和`y`坐标，它将在指定位置绘制一个点：

`scatter_squares.py`

```
import matplotlib.pyplot as plt

plt.scatter(2, 4)
plt.show()
```

下面来设置输出的样式，使其更有趣：添加标题，给轴加上标签，并确保所有文本都大到能够看清：

```
import matplotlib.pyplot as plt
```

```
❶ plt.scatter(2, 4, s=200)
```

```
# 设置图表标题并给坐标轴加上标签
plt.title("Square Numbers", fontsize=24)
plt.xlabel("Value", fontsize=14)
plt.ylabel("Square of Value", fontsize=14)

# 设置刻度标记的大小
plt.tick_params(axis='both', which='major', labelsize=14)

plt.show()
```

在❶处，我们调用了`scatter()`，并使用实参`s`设置了绘制图形时使用的点的尺寸。如果此时运行`scatter_squares.py`，将在图表中央看到一个点，如图15-4所示。

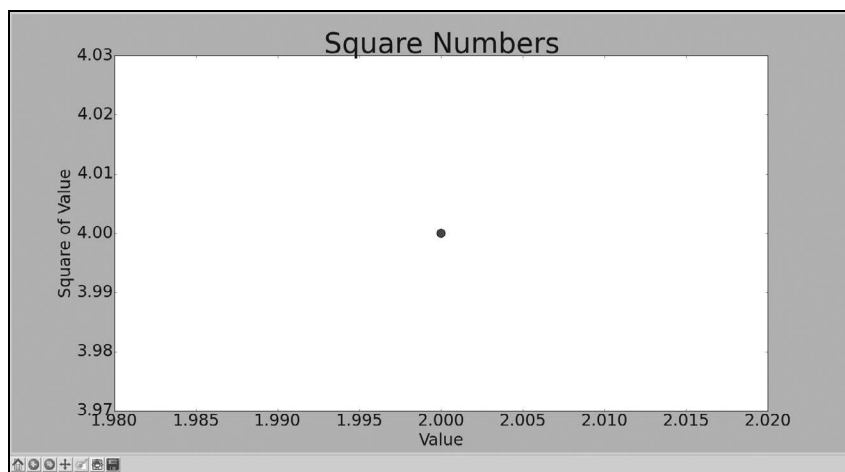


图15-4 绘制单个点

15.2.4 使用 `scatter()` 绘制一系列点

要绘制一系列的点，可向`scatter()`传递两个分别包含`x`值和`y`值的列表，如下所示：

scatter_squares.py

```
import matplotlib.pyplot as plt

x_values = [1, 2, 3, 4, 5]
y_values = [1, 4, 9, 16, 25]

plt.scatter(x_values, y_values, s=100)

# 设置图表标题并给坐标轴指定标签
--snip--
```


列表`x_values`包含要计算其平方值的数字，而列表`y_values`包含前述每个数字的平方值。将这些列表传递给`scatter()`时，`matplotlib`依次从每个列表中读取一个值来绘制一个点。要绘制的点的坐标分别为(1, 1)、(2, 4)、(3, 9)、(4, 16)和(5, 25)，最终的结果如图15-5所示。

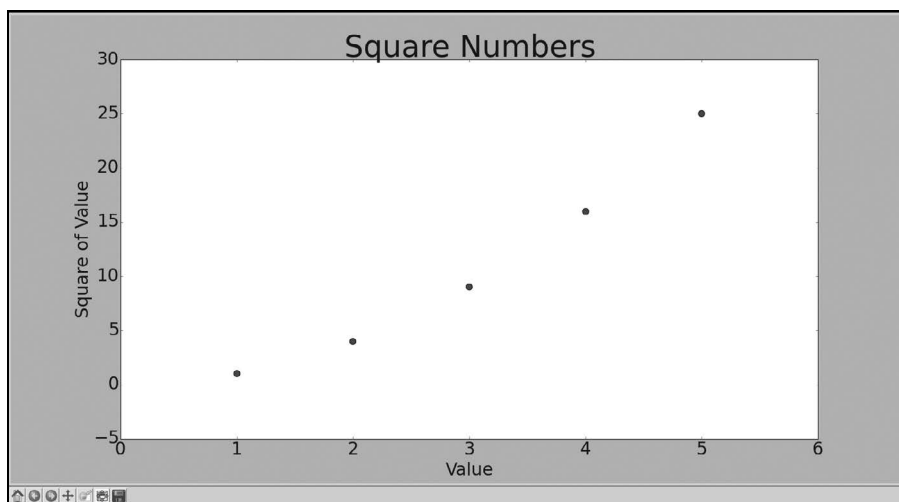


图15-5 由多个点组成的散点图

15.2.5 自动计算数据

手工计算列表要包含的值可能效率低下，需要绘制的点很多时尤其如此。可以不必手工计算包含点坐标的列表，而让Python循环来替我们完成这种计算。下面是绘制1000个点的代码：

scatter_squares.py

```
import matplotlib.pyplot as plt

❶ x_values = list(range(1, 1001))
   y_values = [x**2 for x in x_values]

❷ plt.scatter(x_values, y_values, s=40)

   # 设置图表标题并给坐标轴加上标签
   --snip--

   # 设置每个坐标轴的取值范围
❸ plt.axis([0, 1100, 0, 1100000])

plt.show()
```

我们首先创建了一个包含`x`值的列表，其中包含数字1~1000（见❶）。接下来是一个生成`y`值的列表解析，它遍历`x`值（`for x in x_values`），计算其平方值（`x**2`），并将结果存储到列表`y_values`

中。然后，将输入列表和输出列表传递给`scatter()`（见❷）。

由于这个数据集较大，我们将点设置得较小，并使用函数`axis()`指定了每个坐标轴的取值范围（见❸）。函数`axis()`要求提供四个值：`x`和`y`坐标轴的最小值和最大值。在这里，我们将`x`坐标轴的取值范围设置为0~1100，并将`y`坐标轴的取值范围设置为0~1 000 000。结果如图15-6所示。

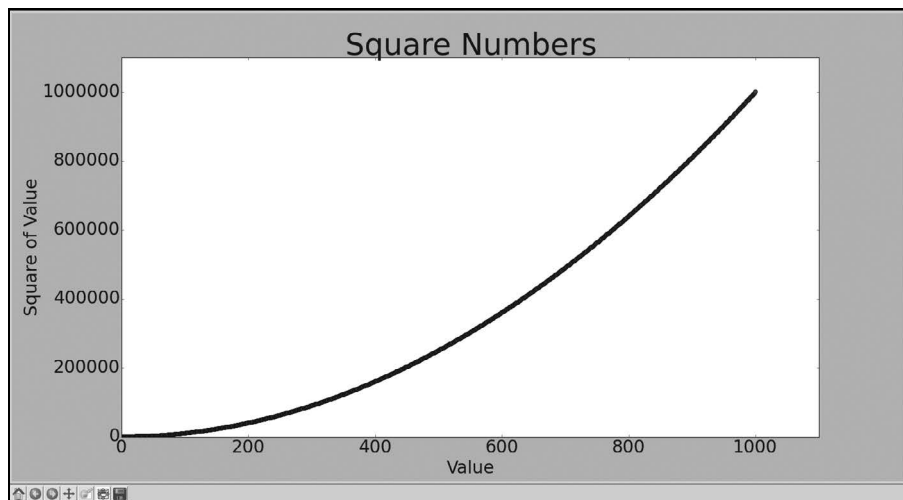


图15-6 Python绘制1000个点与绘制5个点一样容易

15.2.6 删除数据点的轮廓

`matplotlib`允许你给散点图中的各个点指定颜色。默认为蓝色点和黑色轮廓，在散点图包含的数据点不多时效果很好。但绘制很多点时，黑色轮廓可能会粘连在一起。要删除数据点的轮廓，可在调用`scatter()`时传递实参`edgecolor='none'`：

```
plt.scatter(x_values, y_values, edgecolor='none', s=40)
```

将相应调用修改为上述代码后，如果再运行`scatter_squares.py`，在图表中看到的将是蓝色实心点。

15.2.7 自定义颜色

要修改数据点的颜色，可向`scatter()`传递参数`c`，并将其设置为要使用的颜色的名称，如下所示：

```
plt.scatter(x_values, y_values, c='red', edgecolor='none', s=40)
```

你还可以使用RGB颜色模式自定义颜色。要指定自定义颜色，可传递参数`c`，并将其设置为

一个元组，其中包含三个0~1之间的小数值，它们分别表示红色、绿色和蓝色分量。例如，下面的代码行创建一个由淡蓝色点组成的散点图：

```
plt.scatter(x_values, y_values, c=(0, 0, 0.8), edgecolor='none', s=40)
```

值越接近0，指定的颜色越深，值越接近1，指定的颜色越浅。

15.2.8 使用颜色映射

颜色映射（colormap）是一系列颜色，它们从起始颜色渐变到结束颜色。在可视化中，颜色映射用于突出数据的规律，例如，你可能用较浅的颜色来显示较小的值，并使用较深的颜色来显示较大的值。

模块pyplot内置了一组颜色映射。要使用这些颜色映射，你需要告诉pyplot该如何设置数据集中每个点的颜色。下面演示了如何根据每个点的y值来设置其颜色：

scatter_squares.py

```
import matplotlib.pyplot as plt

x_values = list(range(1001))
y_values = [x**2 for x in x_values]

plt.scatter(x_values, y_values, c=y_values, cmap=plt.cm.Blues,
            edgecolor='none', s=40)

# 设置图表标题并给坐标轴加上标签
-- snip --
```

我们将参数c设置成了一个y值列表，并使用参数cmap告诉pyplot使用哪个颜色映射。这些代码将y值较小的点显示为浅蓝色，并将y值较大的点显示为深蓝色，生成的图形如图15-7所示。

注意 要了解pyplot中所有的颜色映射，请访问<http://matplotlib.org/>，单击Examples，向下滚动到Color Examples，再单击colormaps_reference。

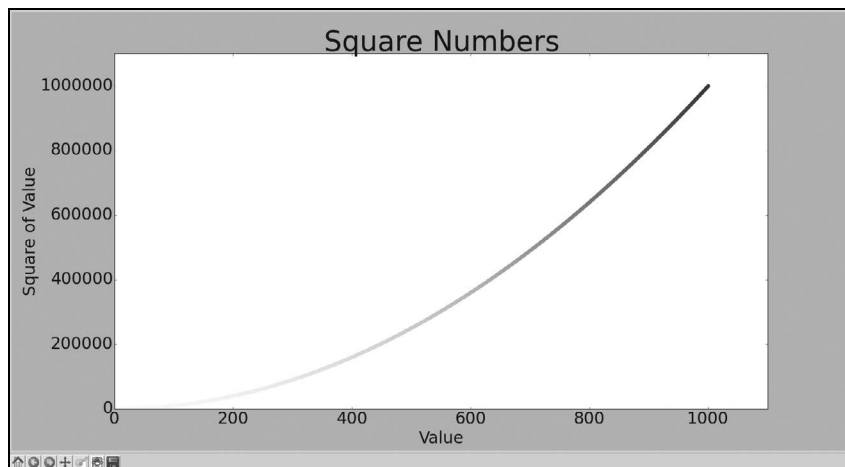


图15-7 使用颜色映射Blues的图表

15.2.9 自动保存图表

要让程序自动将图表保存到文件中，可将对`plt.show()`的调用替换为对`plt.savefig()`的调用：

```
plt.savefig('squares_plot.png', bbox_inches='tight')
```

第一个实参指定要以什么样的文件名保存图表，这个文件将存储到`scatter_squares.py`所在的目录中；第二个实参指定将图表多余的空白区域裁剪掉。如果要保留图表周围多余的空白区域，可省略这个实参。

动手试一试

15-1 立方：数字的三次方被称为其立方。请绘制一个图形，显示前5个整数的立方值，再绘制一个图形，显示前5000个整数的立方值。

15-2 彩色立方：给你前面绘制的立方图指定颜色映射。

15

15.3 随机漫步

在本节中，我们将使用Python来生成随机漫步数据，再使用matplotlib以引人瞩目的方式将这些数据呈现出来。随机漫步是这样行走得到的路径：每次行走都完全是随机的，没有明确的方向，结果是由一系列随机决策决定的。你可以这样认为，随机漫步就是蚂蚁在晕头转向的情况下，每

次都沿随机的方向前行所经过的路径。

在自然界、物理学、生物学、化学和经济领域，随机漫步都有其实用用途。例如，漂浮在水滴上的花粉因不断受到水分子的挤压而在水面上移动。水滴中的分子运动是随机的，因此花粉在水面上的运动路径犹如随机漫步。我们稍后将编写的代码模拟了现实世界的很多情形。

15.3.1 创建 RandomWalk() 类

为模拟随机漫步，我们将创建一个名为RandomWalk的类，它随机地选择前进方向。这个类需要三个属性，其中一个存储随机漫步次数的变量，其他两个是列表，分别存储随机漫步经过的每个点的x和y坐标。

RandomWalk类只包含两个方法：__init__()和fill_walk()，其中后者计算随机漫步经过的所有点。下面先来看看__init__()，如下所示：

random_walk.py

```
❶ from random import choice

class RandomWalk():
    """一个生成随机漫步数据的类"""

    ❷ def __init__(self, num_points=5000):
        """初始化随机漫步的属性"""
        self.num_points = num_points

        # 所有随机漫步都始于(0, 0)
    ❸ self.x_values = [0]
        self.y_values = [0]
```

为做出随机决策，我们将所有可能的选择都存储在一个列表中，并在每次做决策时都使用choice()来决定使用哪种选择（见❶）。接下来，我们将随机漫步包含的默认点数设置为5000，这大到足以生成有趣的模式，同时又足够小，可确保能够快速模拟随机漫步（见❷）。然后，在❸处，我们创建了两个用于存储x和y值的列表，并让每次漫步都从点(0, 0)出发。

15.3.2 选择方向

我们将使用fill_walk()来生成漫步包含的点，并决定每次漫步的方向，如下所示。请将这个方法添加到random_walk.py中：

random_walk.py

```
def fill_walk(self):
    """计算随机漫步包含的所有点"""

    # 不断漫步，直到列表达到指定的长度
    ❶ while len(self.x_values) < self.num_points:
```

```

# 决定前进方向以及沿这个方向前进的距离
❷ x_direction = choice([1, -1])
   x_distance = choice([0, 1, 2, 3, 4])
❸ x_step = x_direction * x_distance

   y_direction = choice([1, -1])
   y_distance = choice([0, 1, 2, 3, 4])
❹ y_step = y_direction * y_distance

# 拒绝原地踏步
❺ if x_step == 0 and y_step == 0:
    continue

# 计算下一个点的x和y值
❻ next_x = self.x_values[-1] + x_step
   next_y = self.y_values[-1] + y_step

self.x_values.append(next_x)
self.y_values.append(next_y)

```

在❶处，我们建立了一个循环，这个循环不断运行，直到漫步包含所需数量的点。这个方法的主要部分告诉Python如何模拟四种漫步决定：向右走还是向左走？沿指定的方向走多远？向上走还是向下走？沿选定的方向走多远？

我们使用`choice([1, -1])`给`x_direction`选择一个值，结果要么是表示向右走的1，要么是表示向左走的-1（见❷）。接下来，`choice([0, 1, 2, 3, 4])`随机地选择一个0~4之间的整数，告诉Python沿指定的方向走多远（`x_distance`）。（通过包含0，我们不仅能够沿两个轴移动，还能够沿y轴移动。）

在❸和❹处，我们将移动方向乘以移动距离，以确定沿x和y轴移动的距离。如果`x_step`为正，将向右移动，为负将向左移动，而为零将垂直移动；如果`y_step`为正，就意味着向上移动，为负意味着向下移动，而为零意味着水平移动。如果`x_step`和`y_step`都为零，则意味着原地踏步，我们拒绝这样的情况，接着执行下一次循环（见❺）。

为获取漫步中下一个点的x值，我们将`x_step`与`x_values`中的最后一个值相加（见❻），对于y值也做相同的处理。获得下一个点的x值和y值后，我们将它们分别附加到列表`x_values`和`y_values`的末尾。

15.3.3 绘制随机漫步图

下面的代码将随机漫步的所有点都绘制出来：

`rw_visual.py`

```

import matplotlib.pyplot as plt

from random_walk import RandomWalk

```

```
# 创建一个RandomWalk实例，并将其包含的点都绘制出来
❶ rw = RandomWalk()
   rw.fill_walk()
❷ plt.scatter(rw.x_values, rw.y_values, s=15)
   plt.show()
```

我们首先导入了模块pyplot和RandomWalk类，然后创建了一个RandomWalk实例，并将其存储到rw中（见❶），再调用fill_walk()。在❷处，我们将随机漫步包含的x和y值传递给scatter()，并选择了合适的点尺寸。图15-8显示了包含5000个点的随机漫步图（本节的示意图未包含matplotlib查看器部分，但你运行rw_visual.py时，依然会看到）。

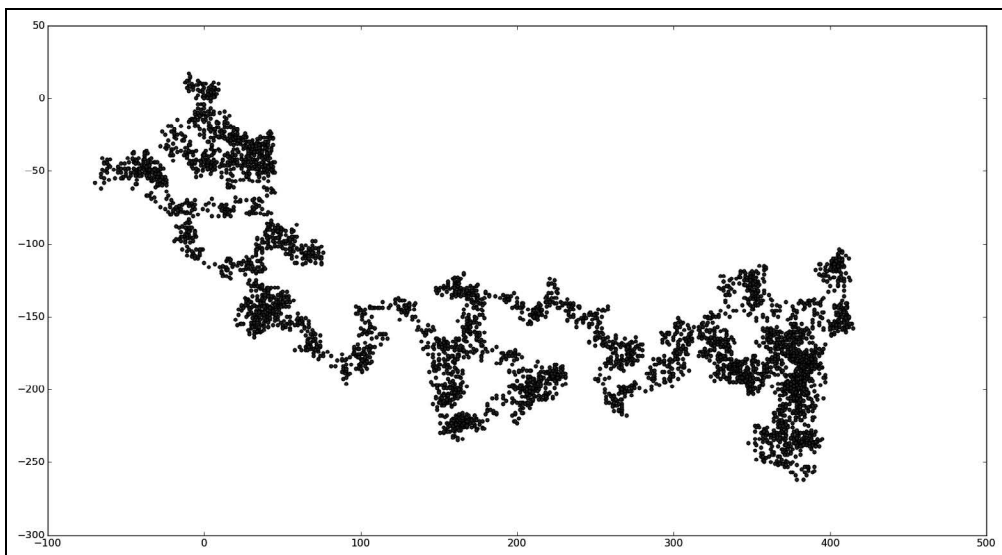


图15-8 包含5000个点的随机漫步

15.3.4 模拟多次随机漫步

每次随机漫步都不同，因此探索可能生成的各种模式很有趣。要在不多次运行程序的情况下使用前面的代码模拟多次随机漫步，一种办法是把这些代码放在一个while循环中，如下所示：

rw_visual.py

```
import matplotlib.pyplot as plt

from random_walk import RandomWalk

# 只要程序处于活动状态，就不断地模拟随机漫步
while True:
    # 创建一个RandomWalk实例，并将其包含的点都绘制出来
    rw = RandomWalk()
```

```

rw.fill_walk()
plt.scatter(rw.x_values, rw.y_values, s=15)
plt.show()

❶ keep_running = input("Make another walk? (y/n): ")
    if keep_running == 'n':
        break

```

这些代码模拟一次随机漫步，在matplotlib查看器中显示结果，再在不关闭查看器的情况下暂停。如果你关闭查看器，程序将询问你是否要再模拟一次随机漫步。如果你输入y，可模拟多次随机漫步：这些随机漫步都在起点附近进行，大多沿特定方向偏离起点，漫步点分布不均匀等。要结束程序，请输入n。

注意 如果你使用的是Python 2.7，别忘了将❶处的input()替换为raw_input()。

15.3.5 设置随机漫步图的样式

在本节中，我们将定制图表，以突出每次漫步的重要特征，并让分散注意力的元素不那么显眼。为此，我们确定要突出的元素，如漫步的起点、终点和经过的路径。接下来确定要使其不那么显眼的元素，如刻度标记和标签。最终的结果是简单的可视化表示，清楚地指出了每次漫步经过的路径。

15.3.6 给点着色

我们将使用颜色映射来指出漫步中各点的先后顺序，并删除每个点的黑色轮廓，让它们的颜色更明显。为根据漫步中各点的先后顺序进行着色，我们传递参数c，并将其设置为一个列表，其中包含各点的先后顺序。由于这些点是按顺序绘制的，因此给参数c指定的列表只需包含数字1~5000，如下所示：

rw_visual.py

```

--snip--
while True:
    # 创建一个RandomWalk实例，并将其包含的点都绘制出来
    rw = RandomWalk()
    rw.fill_walk()

    ❶ point_numbers = list(range(rw.num_points))
    plt.scatter(rw.x_values, rw.y_values, c=point_numbers, cmap=plt.cm.Blues,
                edgecolor='none', s=15)
    plt.show()

    keep_running = input("Make another walk? (y/n): ")
--snip--

```

15

在❶处，我们使用了`range()`生成了一个数字列表，其中包含的数字个数与漫步包含的点数相同。接下来，我们将这个列表存储在`point_numbers`中，以便后面使用它来设置每个漫步点的颜色。我们将参数`c`设置为`point_numbers`，指定使用颜色映射`Blues`，并传递实参`edgecolor=None`以删除每个点周围的轮廓。最终的随机漫步图从浅蓝色渐变为深蓝色，如图15-9所示。

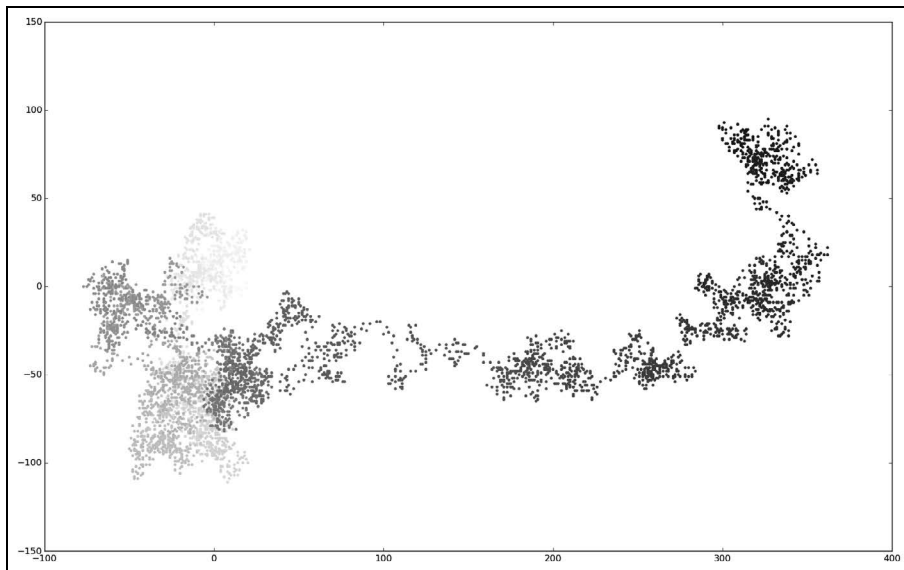


图15-9 使用颜色映射`Blues`着色的随机漫步图

15.3.7 重新绘制起点和终点

除了给随机漫步的各个点着色，以指出它们的先后顺序外，如果还能呈现随机漫步的起点和终点就更好了。为此，可在绘制随机漫步图后重新绘制起点和终点。我们让起点和终点变得更大，并显示为不同的颜色，以突出它们，如下所示：

`rw_visual.py`

```
--snip--
while True:
    --snip--
    plt.scatter(rw.x_values, rw.y_values, c=point_numbers, cmap=plt.cm.Blues,
                edgecolor='none', s=15)

    # 突出起点和终点
    plt.scatter(0, 0, c='green', edgecolor='none', s=100)
    plt.scatter(rw.x_values[-1], rw.y_values[-1], c='red', edgecolor='none',
                s=100)

    plt.show()
```

```
--snip--
```

为突出起点，我们使用绿色绘制点(0, 0)，并使其比其他点大（`s=100`）。为突出终点，我们在漫步包含的最后一个`x`和`y`值处绘制一个点，将其颜色设置为红色，并将尺寸设置为100。请务必将这些代码放在调用`plt.show()`的代码前面，确保在其他点的上面绘制起点和终点。

如果你现在运行这些代码，将能准确地知道每次随机漫步的起点和终点（如果起点和终点不明显，请调整它们的颜色和大小，直到明显为止）。

15.3.8 隐藏坐标轴

下面来隐藏这个图表中的坐标轴，以免我们注意的是坐标轴而不是随机漫步路径。要隐藏坐标轴，可使用如下代码：

```
rw_visual.py
```

```
--snip--
while True:
    --snip--
    plt.scatter(rw.x_values[-1], rw.y_values[-1], c='red', edgecolors='none',
                s=100)

    # 隐藏坐标轴
    ❶ plt.axes().get_xaxis().set_visible(False)
    plt.axes().get_yaxis().set_visible(False)

    plt.show()
--snip--
```

为修改坐标轴，使用了函数`plt.axes()`（见❶）来将每条坐标轴的可见性都设置为`False`。随着你越来越多地进行数据可视化，经常会看到这种串接方法的方式。

如果你现在运行`rw_visual.py`，将看到一系列图形，但看不到坐标轴。

15.3.9 增加点数

下面来增加点数，以提供更多的数据。为此，我们在创建`RandomWalk`实例时增大`num_points`的值，并在绘图时调整每个点的大小，如下所示：

```
rw_visual.py
```

```
--snip--
while True:
    # 创建一个RandomWalk实例，并将其包含的点都绘制出来
    rw = RandomWalk(50000)
    rw.fill_walk()

    # 绘制点并将图形显示出来
    point_numbers = list(range(rw.num_points))
```

```
plt.scatter(rw.x_values, rw.y_values, c=point_numbers, cmap=plt.cm.Blues,  
            edgecolor='none', s=1)  
--snip--
```

这个示例模拟了一次包含50 000个点的随机漫步（以模拟现实情况），并将每个点的大小都设置为1。最终的随机漫步图更纤细，犹如云朵，如图15-10所示。正如你看到的，我们使用简单的散点图制作出了一件艺术品！

请尝试修改上述代码，看看将漫步包含的点数增加到多少后，程序的运行速度变得极其缓慢或绘制出的图形变得很难看。

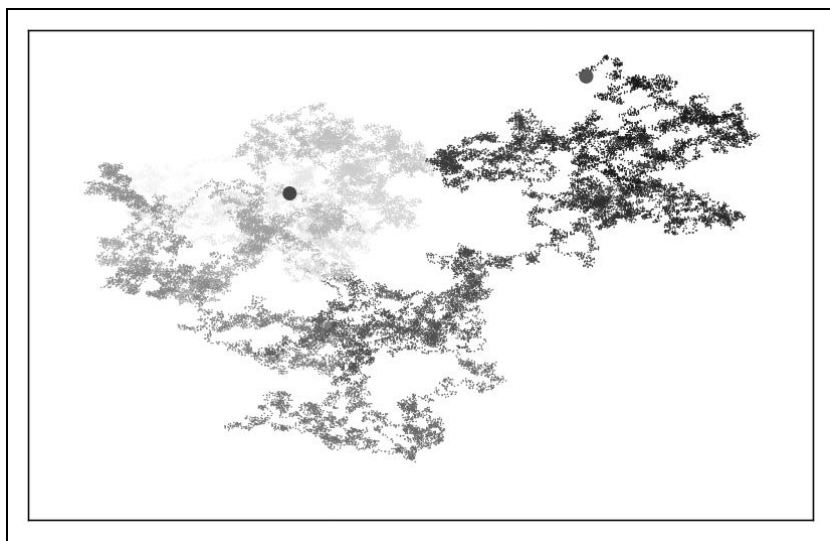


图15-10 包含50 000个点的随机漫步

15.3.10 调整尺寸以适合屏幕

图表适合屏幕大小时，更能有效地将数据中的规律呈现出来。为让绘图窗口更适合屏幕大小，可像下面这样调整matplotlib输出的尺寸：

rw_visual.py

```
--snip--  
while True:  
    # 创建一个RandomWalk实例，并将其包含的点都绘制出来  
    rw = RandomWalk()  
    rw.fill_walk()  
  
    # 设置绘图窗口的尺寸  
    plt.figure(figsize=(10, 6))  
--snip--
```

函数`figure()`用于指定图表的宽度、高度、分辨率和背景色。你需要给形参`figsize`指定一个元组，向`matplotlib`指出绘图窗口的尺寸，单位为英寸。

Python假定屏幕分辨率为80像素/英寸，如果上述代码指定的图表尺寸不合适，可根据需要调整其中的数字。如果你知道自己的系统的分辨率，可使用形参`dpi`向`figure()`传递该分辨率，以有效地利用可用的屏幕空间，如下所示：

```
plt.figure(dpi=128, figsize=(10, 6))
```

动手试一试

15-3 分子运动：修改 `rw_visual.py`，将其中的 `plt.scatter()` 替换为 `plt.plot()`。为模拟花粉在水滴表面的运动路径，向 `plt.plot()` 传递 `rw.x_values` 和 `rw.y_values`，并指定实参值 `linewidth`。使用 5000 个点而不是 50 000 个点。

15-4 改进的随机漫步：在类 `RandomWalk` 中，`x_step` 和 `y_step` 是根据相同的条件生成的：从列表 `[1, -1]` 中随机地选择方向，并从列表 `[0, 1, 2, 3, 4]` 中随机地选择距离。请修改这些列表中的值，看看对随机漫步路径有何影响。尝试使用更长的距离选择列表，如 `0~8`；或者将 `-1` 从 `x` 或 `y` 方向列表中删除。

15-5 重构：方法 `fill_walk()` 很长。请新建一个名为 `get_step()` 的方法，用于确定每次漫步的距离和方向，并计算这次漫步将如何移动。然后，在 `fill_walk()` 中调用 `get_step()` 两次：

```
x_step = get_step()
y_step = get_step()
```

通过这样的重构，可缩小 `fill_walk()` 的规模，让这个方法的阅读和理解起来更容易。

15.4 使用 Pygal 模拟掷骰子

15

在本节中，我们将使用Python可视化包Pygal来生成可缩放的矢量图形文件。对于需要在尺寸不同的屏幕上显示的图表，这很有用，因为它们将自动缩放，以适合观看者的屏幕。如果你打算以在线方式使用图表，请考虑使用Pygal来生成它们，这样它们在任何设备上显示时都会很美观。

在这个项目中，我们将对掷骰子的结果进行分析。掷6面的常规骰子时，可能出现的结果为1~6点，且出现每种结果的可能性相同。然而，如果同时掷两个骰子，某些点数出现的可能性将比其他点数大。为确定哪些点数出现的可能性最大，我们将生成一个表示掷骰子结果的数据集，并根据结果绘制出一个图形。

在数学领域，常常利用掷骰子来解释各种数据分析，但它在赌场和其他博弈场景中也得到了

实际应用，在游戏《大富翁》以及众多角色扮演游戏中亦如此。

15.4.1 安装 Pygal

请使用pip来安装Pygal（如果还未使用过pip，请参阅12.2.1节）。在Linux和OS X系统中，应执行的命令类似于下面这样：

```
pip install --user pygal
```

在Windows系统中，命令类似于下面这样：

```
python -m pip install --user pygal
```

注意 你可能需要使用命令pip3而不是pip，如果这还是不管用，你可能需要删除标志--user。

15.4.2 Pygal 画廊

要了解使用Pygal可创建什么样的图表，请查看图表类型画廊：访问<http://www.pygal.org/>，单击Documentation，再单击Chart types。每个示例都包含源代码，让你知道这些图表是如何生成的。

15.4.3 创建 Die 类

下面的类模拟掷一个骰子：

die.py

```
from random import randint

class Die():
    """表示一个骰子的类"""

    ❶ def __init__(self, num_sides=6):
        """骰子默认为6面"""
        self.num_sides = num_sides

        def roll(self):
            ❷ """返回一个位于1和骰子面数之间的随机值"""
            return randint(1, self.num_sides)
```

方法__init__()接受一个可选参数。创建这个类的实例时，如果没有指定任何实参，面数默认为6；如果指定了实参，这个值将用于设置骰子的面数（见❶）。骰子是根据面数命名的，6面的骰子名为D6，8面的骰子名为D8，以此类推。

方法roll()使用函数randint()来返回一个1和面数之间的随机数（见❷）。这个函数可能返回

起始值1、终止值num_sides或这两个值之间的任何整数。

15.4.4 掷骰子

使用这个类来创建图表前，先来掷D6骰子，将结果打印出来，并检查结果是否合理：

die_visual.py

```
from die import Die

# 创建一个D6
❶ die = Die()

# 掷几次骰子，并将结果存储在一个列表中
results = []
❷ for roll_num in range(100):
    result = die.roll()
    results.append(result)

print(results)
```

在❶处，我们创建了一个Die实例，其面数为默认值6。在❷处，我们掷骰子100次，并将每次的结果都存储在列表results中。下面是一个示例结果集：

```
[4, 6, 5, 6, 1, 5, 6, 3, 5, 3, 5, 3, 2, 2, 1, 3, 1, 5, 3, 6, 3, 6, 5, 4,
 1, 1, 4, 2, 3, 6, 4, 2, 6, 4, 1, 3, 2, 5, 6, 3, 6, 2, 1, 1, 3, 4, 1, 4,
 3, 5, 1, 4, 5, 5, 2, 3, 3, 1, 2, 3, 5, 6, 2, 5, 6, 1, 3, 2, 1, 1, 1, 6,
 5, 5, 2, 2, 6, 4, 1, 4, 5, 1, 1, 1, 4, 5, 3, 3, 1, 3, 5, 4, 5, 6, 5, 4,
 1, 5, 1, 2]
```

通过快速扫描这些结果可知，Die类看起来没有问题。我们见到了值1和6，这表明返回了最大和最小的可能值；我们没有见到0或7，这表明结果都在正确的范围内。我们还看到了1~6的所有数字，这表明所有可能的结果都出现了。

15.4.5 分析结果

为分析掷一个D6骰子的结果，我们计算每个点数出现的次数：

die_visual.py

```
--snip--
# 掷几次骰子，并将结果存储在一个列表中
results = []
❶ for roll_num in range(1000):
    result = die.roll()
    results.append(result)

# 分析结果
frequencies = []
```

```

❷ for value in range(1, die.num_sides+1):
❸     frequency = results.count(value)
❹     frequencies.append(frequency)

print(frequencies)

```

由于我们将使用Pygal来进行分析，而不是将结果打印出来，因此可以将模拟掷骰子的次数增加到1000（见❶）。为分析结果，我们创建了空列表frequencies，用于存储每种点数出现的次数。在❷处，我们遍历可能的点数（这里为1~6），计算每种点数在results中出现了多少次（见❸），并将这个值附加到列表frequencies的末尾（见❹）。接下来，我们在可视化之前将这个列表打印出来：

```
[155, 167, 168, 170, 159, 181]
```

结果看起来是合理的：我们看到了6个值——掷D6骰子时可能出现的每个点数对应一个；我们还发现，没有任何点数出现的频率比其他点数高很多。下面来可视化这些结果。

15.4.6 绘制直方图

有了频率列表后，我们就可以绘制一个表示结果的直方图。直方图是一种条形图，指出了各种结果出现的频率。创建这种直方图的代码如下：

die_visual.py

```

import pygal
--snip--

# 分析结果
frequencies = []
for value in range(1, die.num_sides+1):
    frequency = results.count(value)
    frequencies.append(frequency)

# 对结果进行可视化
❶ hist = pygal.Bar()

    hist.title = "Results of rolling one D6 1000 times."
❷ hist.x_labels = ['1', '2', '3', '4', '5', '6']
    hist.x_title = "Result"
    hist.y_title = "Frequency of Result"

❸ hist.add('D6', frequencies)
    hist.render_to_file('die_visual.svg')

```

为创建条形图，我们创建了一个pygal.Bar()实例，并将其存储在hist中（见❶）。接下来，我们设置hist的属性title（用于标示直方图的字符串），将掷D6骰子的可能结果用作x轴的标签（见❷），并给每个轴都添加了标题。在❸处，我们使用add()将一系列值添加到图表中（向它传递

要给添加的值指定的标签, 还有一个列表, 其中包含将出现在图表中的值)。最后, 我们将这个图表渲染为一个SVG文件, 这种文件的扩展名必须为.svg。

要查看生成的直方图, 最简单的方式是使用Web浏览器。为此, 在任何Web浏览器中新建一个标签页, 再在其中打开文件die_visual.svg (它位于die_visual.py所在的文件夹中)。你将看到一个类似于图15-11所示的图表(为方便印刷, 我稍微修改了这个图表; 默认情况下, Pygal生成的图表的背景比你在图15-11中看到的要暗)。

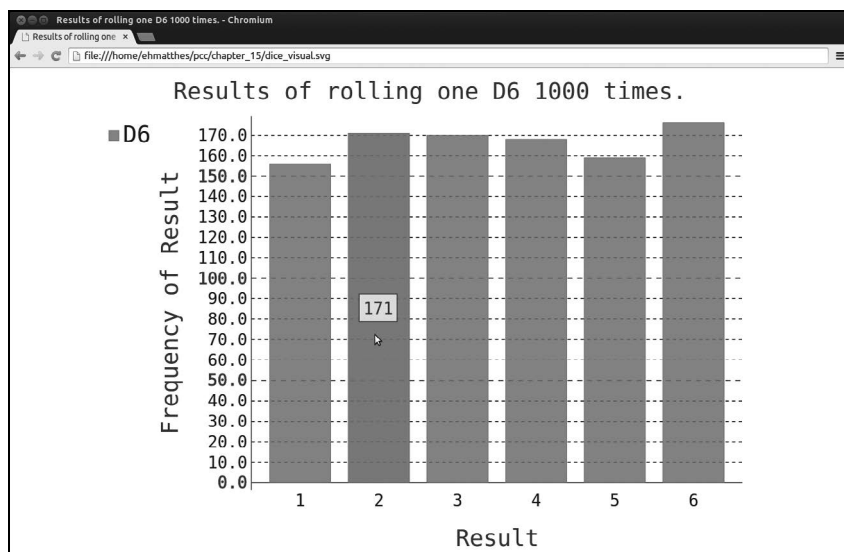


图15-11 使用Pygal创建的简单条形图

注意, Pygal让这个图表具有交互性: 如果你将鼠标指向该图表中的任何条形, 将看到与之相关联的数据。在同一个图表中绘制多个数据集时, 这项功能显得特别有用。

15.4.7 同时掷两个骰子

同时掷两个骰子时, 得到的点数更多, 结果分布情况也不同。下面来修改前面的代码, 创建两个D6骰子, 以模拟同时掷两个骰子的情况。每次掷两个骰子时, 我们都将两个骰子的点数相加, 并将结果存储在results中。请复制die_visual.py并将其保存为dice_visual.py, 再做如下修改:

dice_visual.py

```
import pygal

from die import Die

# 创建两个D6骰子
die_1 = Die()
die_2 = Die()
```



```

# 掷骰子多次，并将结果存储到一个列表中
results = []
for roll_num in range(1000):
❶ result = die_1.roll() + die_2.roll()
    results.append(result)

# 分析结果
frequencies = []
❷ max_result = die_1.num_sides + die_2.num_sides
❸ for value in range(2, max_result+1):
    frequency = results.count(value)
    frequencies.append(frequency)

# 可视化结果
hist = pygal.Bar()

❹ hist.title = "Results of rolling two D6 dice 1000 times."
    hist.x_labels = ['2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12']
    hist.x_title = "Result"
    hist.y_title = "Frequency of Result"

    hist.add('D6 + D6', frequencies)
    hist.render_to_file('dice_visual.svg')

```

创建两个Die实例后，我们掷骰子多次，并计算每次的总点数（见❶）。可能出现的最大点数12为两个骰子的最大可能点数之和，我们将这个值存储在了max_result中（见❷）。可能出现的最小总点数2为两个骰子的最小可能点数之和。分析结果时，我们计算2到max_result的各种点数出现的次数（见❸）。我们原本可以使用range(2, 13)，但这只适用于两个D6骰子。模拟现实世界的情形时，最好编写可轻松地模拟各种情形的代码。前面的代码让我们能够模拟掷任何两个骰子的情形，而不管这些骰子有多少面。

创建图表时，我们修改了标题、x轴标签和数据系列（见❹）。（如果列表x_labels比这里所示的长得多，那么编写一个循环来自动生成它将更合适。）

运行这些代码后，在浏览器中刷新显示图表的标签页，你将看到如图15-12所示的图表。

这个图表显示了掷两个D6骰子时得到的大致结果。正如你看到的，总点数为2或12的可能性最小，而总点数为7的可能性最大，这是因为在6种情况下得到的总点数都为7。这6种情况如下：1和6、2和5、3和4、4和3、5和2、6和1。

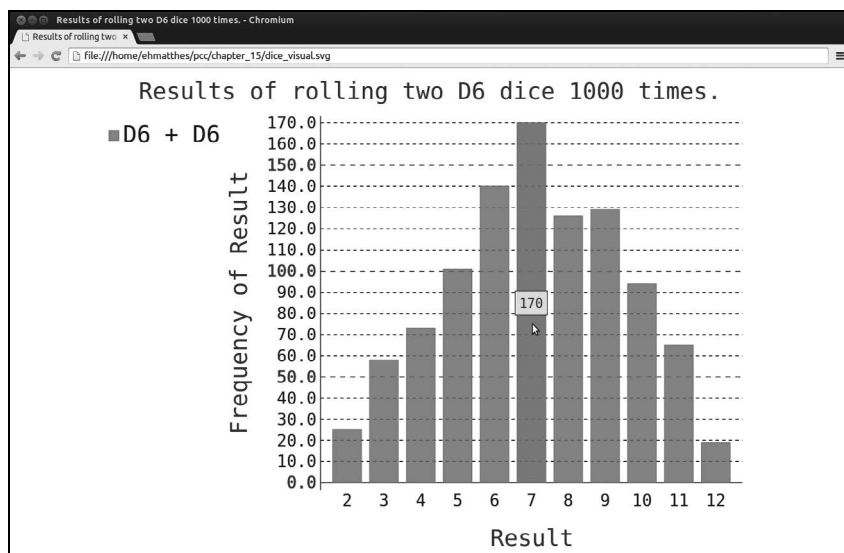


图15-12 模拟同时掷两个6面骰子1000次的结果

15.4.8 同时掷两个面数不同的骰子

下面来创建一个6面骰子和一个10面骰子，看看同时掷这两个骰子50 000次的结果如何：

different_dice.py

```
from die import Die

import pygal

# 创建一个D6和一个D10
die_1 = Die()
❶ die_2 = Die(10)

# 掷骰子多次，并将结果存储在一个列表中
results = []
for roll_num in range(50000):
    result = die_1.roll() + die_2.roll()
    results.append(result)

# 分析结果
--snip--

# 可视化结果
hist = pygal.Bar()

❷ hist.title = "Results of rolling a D6 and a D10 50,000 times."
hist.x_labels = ['2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12',
                 '13', '14', '15', '16']
```

```

hist.x_title = "Result"
hist.y_title = "Frequency of Result"

hist.add('D6 + D10', frequencies)
hist.render_to_file('dice_visual.svg')

```

为创建D10骰子，我们在创建第二个Die实例时传递了实参10（见❶）。我们还修改了第一个循环，以模拟掷骰子50 000次而不是1000次。可能出现的最小总点数依然是2，但现在可能出现的最大总点数为16，因此我们相应地调整了标题、x轴标签和数据系列标签（见❷）。

图15-13显示了最终的图表。可能性最大的点数不是一个，而是5个，这是因为导致出现最小点数和最大点数的组合都只有一种（1和1以及6和10），但面数较小的骰子限制了得到中间点数的组合数：得到总点数7、8、9、10和11的组合数都是6种。因此，这些总点数是最常见的结果，它们出现的可能性相同。

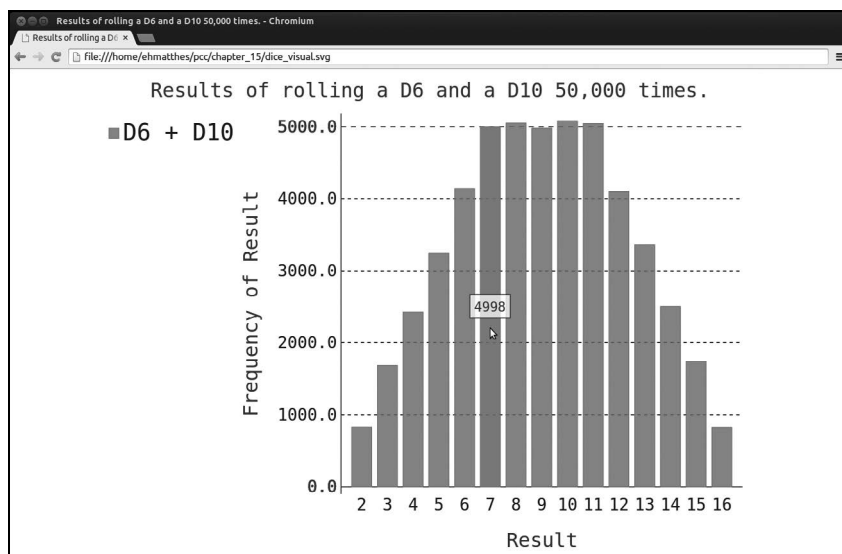


图15-13 同时掷6面骰子和10面骰子50 000次的结果

通过使用Pygal来模拟掷骰子的结果，能够非常自由地探索这种现象。只需几分钟，就可以掷各种骰子很多次。

动手试一试

15-6 自动生成标签：请修改 die.py 和 dice_visual.py，将用来设置 hist.x_labels 值的列表替换为一个自动生成这种列表的循环。如果你熟悉列表解析，可尝试将 die_visual.py 和 dice_visual.py 中的其他 for 循环也替换为列表解析。

15-7 两个 D8 骰子：请模拟同时掷两个 8 面骰子 1000 次的结果。逐渐增加掷骰子的次数，直到系统不堪重负为止。

15-8 同时掷三个骰子：如果你同时掷三个 D6 骰子，可能得到的最小点数为 3，而最大点数为 18。请通过可视化展示同时掷三个 D6 骰子的结果。

15-9 将点数相乘：同时掷两个骰子时，通常将它们的点数相加。请通过可视化展示将两个骰子的点数相乘的结果。

15-10 练习使用本章介绍的两个库：尝试使用 matplotlib 通过可视化来模拟掷骰子的情况，并尝试使用 Pygal 通过可视化来模拟随机漫步的情况。

15.5 小结

在本章中，你学习了：如何生成数据集以及如何对其进行可视化；如何使用matplotlib创建简单的图表，以及如何使用散点图来探索随机漫步过程；如何使用Pygal来创建直方图，以及如何使用直方图来探索同时掷两个面数不同的骰子的结果。

使用代码生成数据集是一种有趣而强大的方式，可用于模拟和探索现实世界的各种情形。完成后面的数据可视化项目时，请注意可使用代码模拟哪些情形。请研究新闻媒体中的可视化，看看其中是否有图表是以你在这些项目中学到的类似方式生成的。

在第16章中，我们将从网上下载数据，并继续使用matplotlib和Pygal来探索这些数据。



在本章中，你将从网上下载数据，并对这些数据进行可视化。网上的数据多得难以置信，且大多未经过仔细检查。如果能够对这些数据进行分析，你就能发现别人没有发现的规律和关联。

我们将访问并可视化以两种常见格式存储的数据：CSV和JSON。我们将使用Python模块csv来处理以CSV（逗号分隔的值）格式存储的天气数据，找出两个不同地区在一段时间内的最高温度和最低温度。然后，我们将使用matplotlib根据下载的数据创建一个图表，展示两个不同地区的气温变化：阿拉斯加锡特卡和加利福尼亚死亡谷。在本章的后面，我们将使用模块json来访问以JSON格式存储的人口

数据，并使用Pygal绘制一幅按国别划分的人口地图。

阅读本章后，你将能够处理各种类型和格式的数据集，并对如何创建复杂的图表有更深入的认识。要处理各种真实世界的数据集，必须能够访问并可视化各种类型和格式的在线数据。

16.1 CSV 文件格式

要在文本文件中存储数据，最简单的方式是将数据作为一系列以逗号分隔的值（CSV）写入文件。这样的文件称为CSV文件。例如，下面是一行CSV格式的天气数据：

```
2014-1-5,61,44,26,18,7,-1,56,30,9,30.34,30.27,30.15,,,10,4,,0.00,0,,195
```

这是阿拉斯加锡特卡2014年1月5日的天气数据，其中包含当天的最高气温和最低气温，还有众多其他数据。CSV文件对人说阅读起来比较麻烦，但程序可轻松地提取并处理其中的值，这有助于加快数据分析过程。

我们将首先处理少量锡特卡的CSV格式的天气数据，这些数据可在本书的配套资源（<https://www.nostarch.com/pythoncrashcourse/>）中找到。请将文件sitka_weather_07-2014.csv复制到存储本程序的文件夹中（下载本书的配套资源后，你就有了这个项目所需的所有文件）。

注意 这个项目使用的天气数据是从<http://www.wunderground.com/history/>下载而来的。

16.1.1 分析 CSV 文件头

csv模块包含在Python标准库中,可用于分析CSV文件中的数据行,让我们能够快速提取感兴趣的值。下面先来查看这个文件的第一行,其中包含一系列有关数据的描述:

highs_lows.py

```
import csv

filename = 'sitka_weather_07-2014.csv'
❶ with open(filename) as f:
❷     reader = csv.reader(f)
❸     header_row = next(reader)
    print(header_row)
```

导入模块csv后,我们将要使用的文件的名称存储在filename中。接下来,我们打开这个文件,并将结果文件对象存储在f中(见❶)。然后,我们调用csv.reader(),并将前面存储的文件对象作为实参传递给它,从而创建一个与该文件相关联的阅读器(reader)对象(见❷)。我们将这个阅读器对象存储在reader中。

模块csv包含函数next(),调用它并将阅读器对象传递给它时,它将返回文件中的下一行。在前面的代码中,我们只调用了next()一次,因此得到的是文件的第一行,其中包含文件头(见❸)。我们将返回的数据存储在header_row中。正如你看到的,header_row包含与天气相关的文件头,指出了每行都包含哪些数据:

```
['AKDT', 'Max TemperatureF', 'Mean TemperatureF', 'Min TemperatureF',
'Max Dew PointF', 'MeanDew PointF', 'Min DewpointF', 'Max Humidity',
'Mean Humidity', 'Min Humidity', 'Max Sea Level PressureIn',
'Mean Sea Level PressureIn', 'Min Sea Level PressureIn',
'Max VisibilityMiles', 'Mean VisibilityMiles', 'Min VisibilityMiles',
'Max Wind SpeedMPH', 'Mean Wind SpeedMPH', 'Max Gust SpeedMPH',
'PrecipitationIn', 'CloudCover', 'Events', 'WindDirDegrees']
```

reader处理文件中以逗号分隔的第一行数据,并将每项数据都作为一个元素存储在列表中。文件头AKDT表示阿拉斯加时间(Alaska Daylight Time),其位置表明每行的第一个值都是日期或时间。文件头Max TemperatureF指出每行的第二个值都是当天的最高华氏温度。可通过阅读其他的文件头来确定文件包含的信息类型。

注意 文件头的格式并非总是一致的,空格和单位可能出现在奇怪的地方。这在原始数据文件中很常见,但不会带来任何问题。

16.1.2 打印文件头及其位置

为让文件头数据更容易理解，将列表中的每个文件头及其位置打印出来：

highs_lows.py

```
--snip--
with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)

❶ for index, column_header in enumerate(header_row):
    print(index, column_header)
```

我们对列表调用了`enumerate()`（见❶）来获取每个元素的索引及其值。（请注意，我们删除了代码行`print(header_row)`，转而显示这个更详细的版本。）

输出如下，其中指出了每个文件头的索引：

```
0 AKDT
1 Max TemperatureF
2 Mean TemperatureF
3 Min TemperatureF
--snip--
20 CloudCover
21 Events
22 WindDirDegrees
```

从中可知，日期和最高气温分别存储在第0列和第1列。为研究这些数据，我们将处理`sitka_weather_07-2014.csv`中的每行数据，并提取其中索引为0和1的值。

16.1.3 提取并读取数据

知道需要哪些列中的数据后，我们来读取一些数据。首先读取每天的最高气温：

highs_lows.py

```
import csv

# 从文件中获取最高气温
filename = 'sitka_weather_07-2014.csv'
with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)

❶ highs = []
❷ for row in reader:
❸     highs.append(row[1])

print(highs)
```

我们创建了一个名为highs的空列表（见❶），再遍历文件中余下的各行（见❷）。阅读器对象从其停留的地方继续往下读取CSV文件，每次都自动返回当前所处位置的下一行。由于我们已经读取了文件头行，这个循环将从第二行开始——从这行开始包含的是实际数据。每次执行该循环时，我们都将索引1处（第2列）的数据附加到highs末尾（见❸）。

下面显示了highs现在存储的数据：

```
['64', '71', '64', '59', '69', '62', '61', '55', '57', '61', '57', '59', '57',
 '61', '64', '61', '59', '63', '60', '57', '69', '63', '62', '59', '57', '57',
 '61', '59', '61', '61', '66']
```

我们提取了每天的最高气温，并将它们作为字符串整洁地存储在一个列表中。

下面使用int()将这些字符串转换为数字，让matplotlib能够读取它们：

highs_lows.py

```
--snip--
highs = []
for row in reader:
❶    high = int(row[1])
    highs.append(high)

print(highs)
```

在❶处，我们将表示气温的字符串转换成了数字，再将其附加到列表末尾。这样，最终的列表将包含以数字表示的每日最高气温：

```
[64, 71, 64, 59, 69, 62, 61, 55, 57, 61, 57, 59, 57, 61, 64, 61, 59, 63, 60, 57,
 69, 63, 62, 59, 57, 57, 61, 59, 61, 61, 66]
```

下面来对这些数据进行可视化。

16.1.4 绘制气温图表

为可视化这些气温数据，我们首先使用matplotlib创建一个显示每日最高气温的简单图形，如下所示：

highs_lows.py

```
import csv

from matplotlib import pyplot as plt

# 从文件中获取最高气温
--snip--

# 根据数据绘制图形
fig = plt.figure(dpi=128, figsize=(10, 6))
❶ plt.plot(highs, c='red')
```

```

# 设置图形的格式
❷ plt.title("Daily high temperatures, July 2014", fontsize=24)
❸ plt.xlabel('', fontsize=16)
plt.ylabel("Temperature (F)", fontsize=16)
plt.tick_params(axis='both', which='major', labelsize=16)

plt.show()

```

我们将最高气温列表传给plot()（见❶），并传递c='red'以便将数据点绘制为红色（红色显示最高气温，蓝色显示最低气温）。接下来，我们设置了一些其他的格式，如字体大小和标签（见❷），这些都在第15章介绍过。鉴于我们还没有添加日期，因此没有给x轴添加标签，但plt.xlabel()确实修改了字体大小，让默认标签更容易看清。图16-1显示了绘制的图表：一个简单的折线图，显示了阿拉斯加锡特卡2014年7月每天的最高气温。

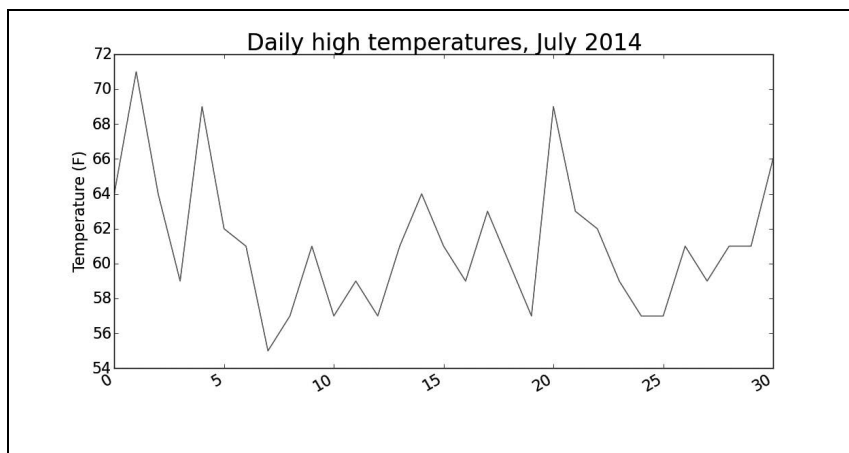


图16-1 阿拉斯加锡特卡2014年7月每日最高气温折线图

16.1.5 模块 datetime

下面在图表中添加日期，使其更有用。在天气数据文件中，第一个日期在第二行：

```
2014-7-1,64,56,50,53,51,48,96,83,58,30.19,--snip--
```

读取该数据时，获得的是一个字符串，因为我们需要想办法将字符串'2014-7-1'转换为一个表示相应日期的对象。为创建一个表示2014年7月1日的对象，可使用模块datetime中的方法strptime()。我们在终端会话中看看strptime()的工作原理：

```

>>> from datetime import datetime
>>> first_date = datetime.strptime('2014-7-1', '%Y-%m-%d')
>>> print(first_date)
2014-07-01 00:00:00

```

我们首先导入了模块datetime中的datetime类，然后调用方法strptime()，并将包含所需日期的字符串作为第一个实参。第二个实参告诉Python如何设置日期的格式。在这个示例中，'%Y-'让Python将字符串中第一个连字符前面的部分视为四位的年份；'%m-'让Python将第二个连字符前面的部分视为表示月份的数字；而'%d'让Python将字符串的最后一部分视为月份中的一天（1~31）。

方法strptime()可接受各种实参，并根据它们来决定如何解读日期。表16-1列出了其中一些这样的实参。

表16-1 模块datetime中设置日期和时间格式的实参

实 参	含 义
%A	星期的名称，如Monday
%B	月份名，如January
%m	用数字表示的月份（01~12）
%d	用数字表示月份中的一天（01~31）
%Y	四位的年份，如2015
%y	两位的年份，如15
%H	24小时制的小时数（00~23）
%I	12小时制的小时数（01~12）
%p	am或pm
%M	分钟数（00~59）
%S	秒数（00~61）

16.1.6 在图表中添加日期

知道如何处理CSV文件中的日期后，就可对气温图形进行改进了，即提取日期和最高气温，并将它们传递给plot()，如下所示：

highs_lows.py

```
import csv
from datetime import datetime

from matplotlib import pyplot as plt

# 从文件中获取日期和最高气温
filename = 'sitka_weather_07-2014.csv'
with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)

❶  dates, highs = [], []
    for row in reader:
❷  current_date = datetime.strptime(row[0], "%Y-%m-%d")
        dates.append(current_date)
```

```

        high = int(row[1])
        highs.append(high)

    # 根据数据绘制图形
    fig = plt.figure(dpi=128, figsize=(10, 6))
    ❸ plt.plot(dates, highs, c='red')

    # 设置图形的格式
    plt.title("Daily high temperatures, July 2014", fontsize=24)
    plt.xlabel('', fontsize=16)
    ❹ fig.autofmt_xdate()
    plt.ylabel("Temperature (F)", fontsize=16)
    plt.tick_params(axis='both', which='major', labelsize=16)

    plt.show()

```

我们创建了两个空列表，用于存储从文件中提取的日期和最高气温（见❶）。然后，我们将包含日期信息的数据（row[0]）转换为datetime对象（见❷），并将其附加到列表dates末尾。在❸处，我们将日期和最高气温值传递给plot()。在❹处，我们调用了fig.autofmt_xdate()来绘制斜的日期标签，以免它们彼此重叠。图16-2显示了改进后的图表。

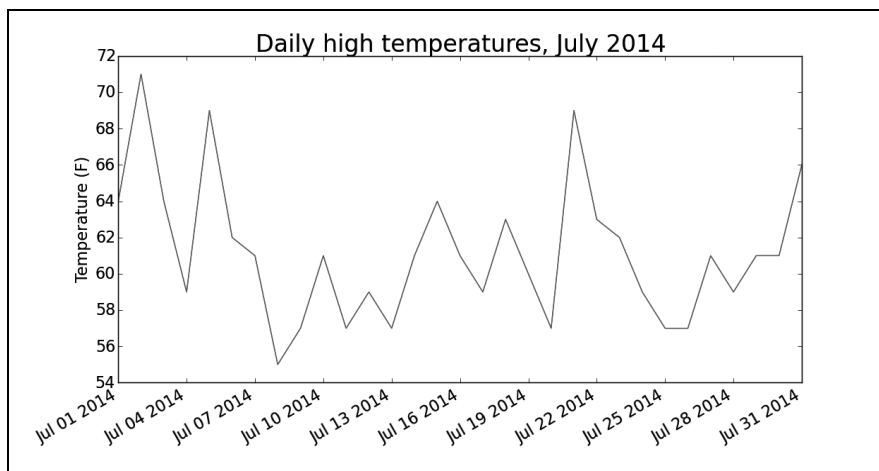


图16-2 现在图表的x轴上有日期，含义更丰富

16.1.7 涵盖更长的时间

设置好图表后，我们来添加更多的数据，以成一幅更复杂的锡特卡天气图。请将文件sitka_weather_2014.csv复制到存储本程序的文件夹中，该文件包含Weather Underground提供的整年的锡特卡天气数据。

现在可以创建覆盖整年的天气图了：

highs_lows.py

```
--snip--
# 从文件中获取日期和最高气温
❶ filename = 'sitka_weather_2014.csv'
with open(filename) as f:
    --snip--
    # 设置图形的格式
❷ plt.title("Daily high temperatures - 2014", fontsize=24)
    plt.xlabel('', fontsize=16)
--snip--
```

我们修改了文件名，以使用新的数据文件sitka_weather_2014.csv（见❶）；我们还修改了图表的标题，以反映其内容的变化（见❷）。图16-3显示了生成的图形。

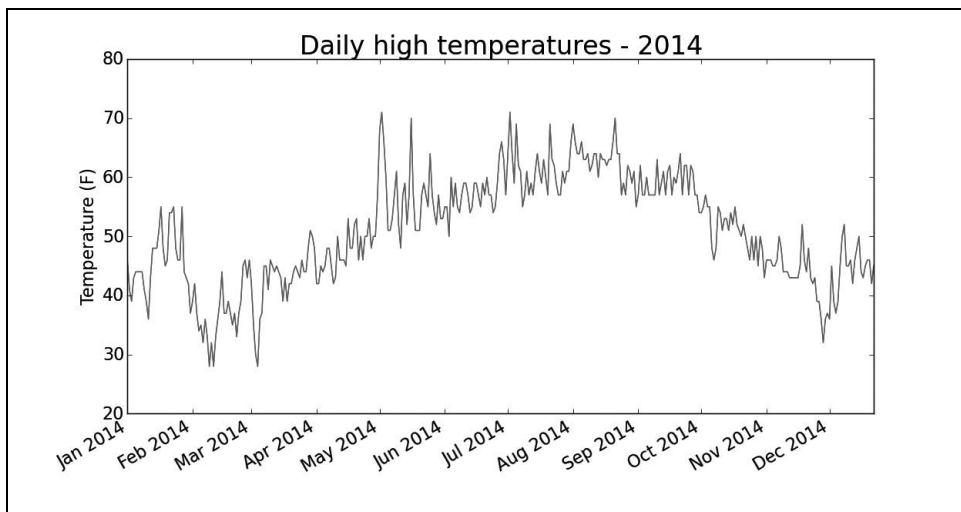


图16-3 一年的天气数据

16.1.8 再绘制一个数据系列

图16-3所示的改进后的图表显示了大量意义深远的数​​据，但我们可以再在其中再添加最低气温数据，使其更有用。为此，需要从数据文件中提取最低气温，并将它们添加到图表中，如下所示：

highs_lows.py

```
--snip--
# 从文件中获取日期、最高气温和最低气温
filename = 'sitka_weather_2014.csv'
with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)
```

```

❶  dates, highs, lows = [], [], []
    for row in reader:
        current_date = datetime.strptime(row[0], "%Y-%m-%d")
        dates.append(current_date)

        high = int(row[1])
        highs.append(high)

❷  low = int(row[3])
    lows.append(low)
    # 根据数据绘制图形
    fig = plt.figure(dpi=128, figsize=(10, 6))
    plt.plot(dates, highs, c='red')
❸  plt.plot(dates, lows, c='blue')

    # 设置图形的格式
❹  plt.title("Daily high and low temperatures - 2014", fontsize=24)
    --snip--

```

在❶处，我们添加了空列表lows，用于存储最低气温。接下来，我们从每行的第4列（row[3]）提取每天的最低气温，并存储它们（见❷）。在❸处，我们添加了一个对plot()的调用，以使用蓝色绘制最低气温。最后，我们修改了标题（见❹）。图16-4显示了这样绘制出来的图表。

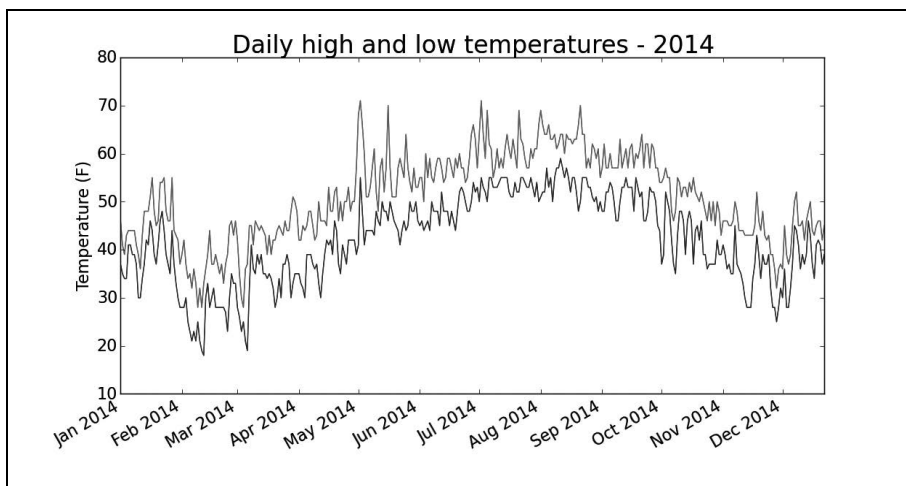


图16-4 在一个图表中包含两个数据系列

16.1.9 给图表区域着色

添加两个数据系列后，我们就可以了解每天的气温范围了。下面来给这个图表做最后的修饰，通过着色来呈现每天的气温范围。为此，我们将使用方法fill_between()，它接受一个x值系列和两个y值系列，并填充两个y值系列之间的空间：

highs_lows.py

```
--snip--
# 根据数据绘制图形
fig = plt.figure(dpi=128, figsize=(10, 6))
❶ plt.plot(dates, highs, c='red', alpha=0.5)
   plt.plot(dates, lows, c='blue', alpha=0.5)
❷ plt.fill_between(dates, highs, lows, facecolor='blue', alpha=0.1)
--snip--
```

❶处的实参alpha指定颜色的透明度。Alpha值为0表示完全透明，1（默认设置）表示完全不透明。通过将alpha设置为0.5，可让红色和蓝色折线的颜色看起来更浅。

在❷处，我们向fill_between()传递了一个x值系列：列表dates，还传递了两个y值系列：highs和lows。实参facecolor指定了填充区域的颜色，我们还将alpha设置成了较小的值0.1，让填充区域将两个数据系列连接起来的同时不分散观察者的注意力。图16-5显示了最高气温和最低气温之间的区域被填充的图表。

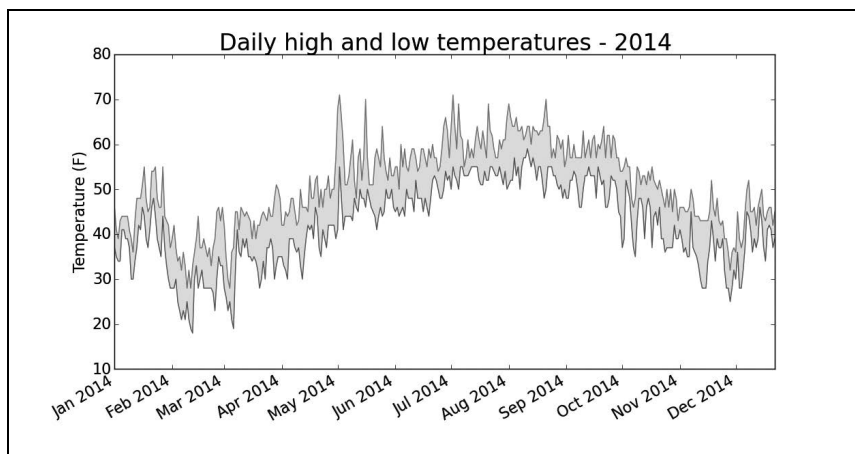


图16-5 给两个数据集之间的区域着色

通过着色，让两个数据集之间的区域显而易见。

16.1.10 错误检查

我们应该能够使用有关任何地方的天气数据来运行highs_lows.py中的代码，但有些气象站会偶尔出现故障，未能收集部分或全部其应该收集的数据。缺失数据可能会引发异常，如果不妥善地处理，还可能导致程序崩溃。

例如，我们来看看生成加利福尼亚死亡谷的气温图时出现的情况。将文件death_valley_2014.csv复制到本程序所在的文件夹，再修改highs_lows.py，使其生成死亡谷的气温图：

highs_lows.py

```
--snip--
# 从文件中获取日期、最高气温和最低气温
filename = 'death_valley_2014.csv'
with open(filename) as f:
--snip--
```

运行这个程序时，出现了一个错误，如下述输出的最后一行所示：

```
Traceback (most recent call last):
  File "highs_lows.py", line 17, in <module>
    high = int(row[1])
ValueError: invalid literal for int() with base 10: ''
```

该traceback指出，Python无法处理其中一天的最高气温，因为它无法将空字符串（''）转换为整数。只要看一下death_valley_2014.csv，就能发现其中的问题：

```
2014-2-16,,,,,,,,,,,,,,,,,0.00,,,-1
```

其中好像没有记录2014年2月16日的数据，表示最高温度的字符串为空。为解决这种问题，我们在从CSV文件中读取值时执行错误检查代码，对分析数据集时可能出现的异常进行处理，如下所示：

highs_lows.py

```
--snip--
# 从文件中获取日期、最高气温和最低气温
filename = 'death_valley_2014.csv'
with open(filename) as f:
    reader = csv.reader(f)
    header_row = next(reader)

    dates, highs, lows = [], [], []
    for row in reader:
        ❶ try:
            current_date = datetime.strptime(row[0], "%Y-%m-%d")
            high = int(row[1])
            low = int(row[3])
        except ValueError:
            ❷ print(current_date, 'missing data')
        else:
            ❸ dates.append(current_date)
            highs.append(high)
            lows.append(low)

#根据数据绘制图形
--snip--

#设置图形的格式
```

```
❶ title = "Daily high and low temperatures - 2014\nDeath Valley, CA"
plt.title(title, fontsize=20)
-- snip --
```

对于每一行，我们都尝试从中提取日期、最高气温和最低气温（见❶）。只要缺失其中一项数据，Python就会引发ValueError异常，而我们可这样处理：打印一条错误消息，指出缺失数据的日期（见❷）。打印错误消息后，循环将接着处理下一行。如果获取特定日期的所有数据时没有发生错误，将运行else代码块，并将数据附加到相应列表的末尾（见❸）。鉴于我们绘图时使用的是有关另一个地方的信息，我们修改了标题，在图表中指出了这个地方（见❹）。

如果你现在运行highs_lows.py，将发现缺失数据的日期只有一个：

```
2014-02-16 missing data
```

图16-6显示了绘制出的图形。

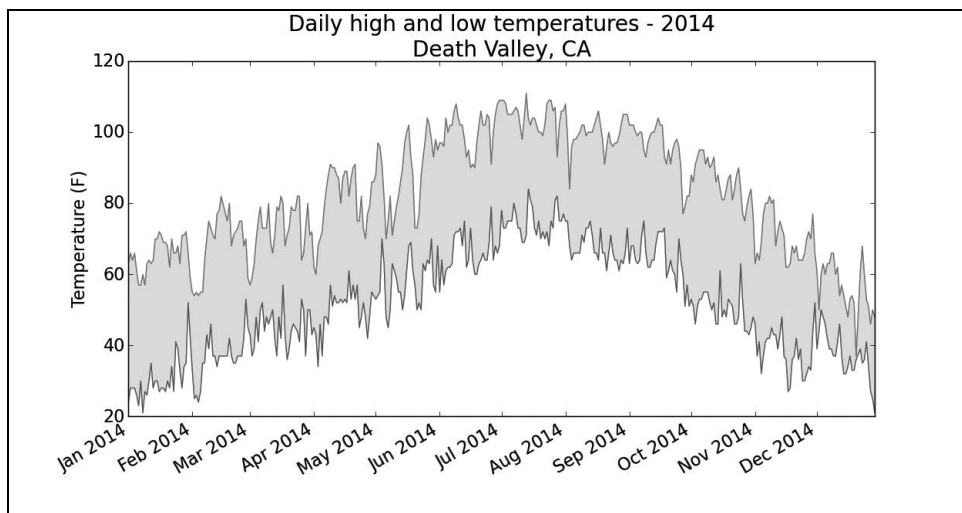


图16-6 死亡谷每日最高气温和最低气温

将这个图表与锡特卡的图表对比可知，总体而言，死亡谷比阿拉斯加东南部暖和，这可能符合预期，但这个沙漠中每天的温差也更大，从着色区域的高度可以明显看出这一点。

使用的很多数据集都可能缺失数据、数据格式不正确或数据本身不正确。对于这样的情形，可使用本书前半部分介绍的工具来处理。在这里，我们使用了一个try-except-else代码块来处理数据缺失的问题。在有些情况下，需要使用continue来跳过一些数据，或者使用remove()或del将已提取的数据删除。可采用任何管用的方法，只要能进行精确而有意义的可视化就好。

动手试一试

16-1 旧金山：旧金山的气温更接近于锡特卡还是死亡谷呢？请绘制一个显示旧金山最高气温和最低气温的图表，并进行比较。可从 <http://www.wunderground.com/history/> 下载几乎任何地方的天气数据。为此，请输入相应的地方和日期范围，滚动到页面底部，找到名为 Comma-Delimited File 的链接，再单击该链接，将数据存储为 CSV 文件。

16-2 比较锡特卡和死亡谷的气温：在有关锡特卡和死亡谷的图表中，气温刻度反映了数据范围的不同。为准确地比较锡特卡和死亡谷的气温范围，需要在 y 轴上使用相同的刻度。为此，请修改图 16-5 和图 16-6 所示图表的 y 轴设置，对锡特卡和死亡谷的气温范围进行直接比较（你也可以对任何两个地方的气温范围进行比较）。你还可以尝试在一个图表中呈现这两个数据集。

16-3 降雨量：选择你感兴趣的任何地方，通过可视化将其降雨量呈现出来。为此，可先只涵盖一个月的数据，确定代码正确无误后，再使用一整年的数据来运行它。

16-4 探索：生成一些图表，对你好奇的任何地方的其他天气数据进行研究。

16.2 制作世界人口地图：JSON 格式

在本节中，你将下载JSON格式的人口数据，并使用json模块来处理它们。Pygal提供了一个适合初学者使用的地图创建工具，你将使用它来对人口数据进行可视化，以探索全球人口的分布情况。

16.2.1 下载世界人口数据

将文件population_data.json复制到本程序所在的文件夹中，这个文件包含全球大部分国家1960~2010年的人口数据。Open Knowledge Foundation（<http://data.okfn.org/>）提供了大量可以免费使用的数据集，这些数据就来自其中一个数据集。

16.2.2 提取相关的数据

我们来研究一下population_data.json，看看如何着手处理这个文件中的数据：

population_data.json

```
[
{
    "Country Name": "Arab World",
    "Country Code": "ARB",
    "Year": "1960",
    "Value": "96388069"
},
```

```
{
  "Country Name": "Arab World",
  "Country Code": "ARB",
  "Year": "1961",
  "Value": "98882541.4"
},
--snip--
]
```

这个文件实际上就是一个很长的Python列表，其中每个元素都是一个包含四个键的字典：国家名、国别码、年份以及表示人口数量的值。我们只关心每个国家2010年的人口数量，因此我们首先编写一个打印这些信息的程序：

world_population.py

```
import json

# 将数据加载到一个列表中
filename = 'population_data.json'
with open(filename) as f:
❶   pop_data = json.load(f)

# 打印每个国家2010年的人口数量
❷ for pop_dict in pop_data:
❸     if pop_dict['Year'] == '2010':
❹         country_name = pop_dict['Country Name']
            population = pop_dict['Value']
            print(country_name + ": " + population)
```

我们首先导入了模块json，以便能够正确地加载文件中的数据，然后，我们将数据存储在pop_data中（见❶）。函数json.load()将数据转换为Python能够处理的格式，这里是一个列表。在❷处，我们遍历pop_data中的每个元素。每个元素都是一个字典，包含四个键-值对，我们将每个字典依次存储在pop_dict中。

在❸处，我们检查字典的'Year'键对应的值是否是2010（由于population_data.json中的值都是用引号括起的，因此我们执行的是字符串比较）。如果年份为2010，我们就将与'Country Name'相关联的值存储到country_name中，并将与'Value'相关联的值存储在population中（见❹）。接下来，我们打印每个国家的名称和人口数量。

输出为一系列国家的名称和人口数量：

```
Arab World: 357868000
Caribbean small states: 6880000
East Asia & Pacific (all income levels): 2201536674
--snip--
Zimbabwe: 12571000
```

我们捕获的数据并非都包含准确的国家名，但这开了一个好头。现在，我们需要将数据转换

为Pygal能够处理的格式。

16.2.3 将字符串转换为数字值

population_data.json中的每个键和值都是字符串。为处理这些人口数据，我们需要将表示人口数量的字符串转换为数字值，为此我们使用函数int()：

world_population.py

```
--snip--
for pop_dict in pop_data:
    if pop_dict['Year'] == '2010':
        country_name = pop_dict['Country Name']
        ❶ population = int(pop_dict['Value'])
        ❷ print(country_name + ": " + str(population))
```

在❶处，我们将每个人口数量值都存储为数字格式。打印人口数量值时，需要将其转换为字符串（见❷）。

然而，对于有些值，这种转换会导致错误，如下所示：

```
Arab World: 357868000
Caribbean small states: 6880000
East Asia & Pacific (all income levels): 2201536674
--snip--
Traceback (most recent call last):
  File "print_populations.py", line 12, in <module>
    population = int(pop_dict['Value'])
  ❶ ValueError: invalid literal for int() with base 10: '1127437398.85751'
```

原始数据的格式常常不统一，因此经常会出现错误。导致上述错误的原因是，Python不能直接将包含小数点的字符串'1127437398.85751'转换为整数（这个小数值可能是人口数据缺失时通过插值得到的）。为消除这种错误，我们先将字符串转换为浮点数，再将浮点数转换为整数：

world_population.py

```
--snip--
for pop_dict in pop_data:
    if pop_dict['Year'] == '2010':
        country = pop_dict['Country Name']
        population = int(float(pop_dict['Value']))
        print(country + ": " + str(population))
```

函数float()将字符串转换为小数，而函数int()丢弃小数部分，返回一个整数。现在，我们可以打印2010年的完整人口数据，不会导致错误了：

```
Arab World: 357868000
Caribbean small states: 6880000
East Asia & Pacific (all income levels): 2201536674
```

```
--snip--
Zimbabwe: 12571000
```

每个字符串都成功地转换成了浮点数，再转换为整数。以数字格式存储人口数量值后，就可以使用它们来制作世界人口地图了。

16.2.4 获取两个字母的国别码

制作地图前，还需要解决数据存在的最后一个问题。Pygal中的地图制作工具要求数据为特定的格式：用国别码表示国家，以及用数字表示人口数量。处理地理政治数据时，经常需要用到几个标准化国别码集。population_data.json中包含的是三个字母的国别码，但Pygal使用两个字母的国别码。我们需要想办法根据国家名获取两个字母的国别码。

Pygal使用的国别码存储在模块i18n（internationalization的缩写）中。字典COUNTRIES包含的键和值分别为两个字母的国别码和国家名。要查看这些国别码，可从模块i18n中导入这个字典，并打印其键和值：

countries.py

```
from pygal.i18n import COUNTRIES

❶ for country_code in sorted(COUNTRIES.keys()):
    print(country_code, COUNTRIES[country_code])
```

在上面的for循环中，我们让Python将键按字母顺序排序（见❶），然后打印每个国别码及其对应的国家：

```
ad Andorra
ae United Arab Emirates
af Afghanistan
--snip--
zw Zimbabwe
```

为获取国别码，我们将编写一个函数，它在COUNTRIES中查找并返回国别码。我们将这个函数放在一个名为country_codes的模块中，以便能够在可视化程序中导入它：

country_codes.py

```
from pygal.i18n import COUNTRIES

❶ def get_country_code(country_name):
    """根据指定的国家，返回Pygal使用的两个字母的国别码"""
    ❷ for code, name in COUNTRIES.items():
        ❸ if name == country_name:
            return code
    # 如果没有找到指定的国家，就返回None
    ❹ return None
```

```
print(get_country_code('Andorra'))
print(get_country_code('United Arab Emirates'))
print(get_country_code('Afghanistan'))
```

`get_country_code()`接受国家名,并将其存储在形参`country_name`中(见❶)。接下来,我们遍历`COUNTRIES`中的国家名-国别码对(见❷);如果找到指定的国家名,就返回相应的国别码(见❸)。在循环后面,我们在没有找到指定的国家名时返回`None`(见❹)。最后,我们使用了三个国家名来调用这个函数,以核实它能否正确地工作。与预期的一样,这个程序输出了三个由两个字母组成的国别码:

```
ad
ae
af
```

使用这个函数前,先将`country_codes.py`中的`print`语句删除。

接下来,在`world_population.py`中导入`get_country_code`:

`world_population.py`

```
import json

from country_codes import get_country_code
--snip--

# 打印每个国家2010年的人口数量
for pop_dict in pop_data:
    if pop_dict['Year'] == '2010':
        country_name = pop_dict['Country Name']
        population = int(float(pop_dict['Value']))
❶    code = get_country_code(country_name)
        if code:
❷        print(code + ": " + str(population))
❸    else:
        print('ERROR - ' + country_name)
```

提取国家名和人口数量后,我们将国别码存储在`code`中,如果没有国别码,就在其中存储`None`(见❶)。如果返回了国别码,就打印国别码和相应国家的人口数量(见❷)。如果没有找到国别码,就显示一条错误消息,其中包含无法找到国别码的国家的名称(见❸)。如果你运行这个程序,将看到一些国别码和相应国家的人口数量,还有一些错误消息:

```
ERROR - Arab World
ERROR - Caribbean small states
ERROR - East Asia & Pacific (all income levels)
--snip--
af: 34385000
al: 3205000
dz: 35468000
--snip--
```

```
ERROR - Yemen, Rep.
zm: 12927000
zw: 12571000
```

导致显示错误消息的原因有两个。首先，并非所有人口数量对应的都是国家，有些人口数量对应的是地区（阿拉伯世界）和经济类群（所有收入水平）。其次，有些统计数据使用了不同的完整国家名（如Yemen, Rep.，而不是Yemen）。当前，我们将忽略导致错误的数据，看看根据成功恢复了的数据制作出的地图是什么样的。

16.2.5 制作世界地图

有了国别码后，制作世界地图易如反掌。Pygal提供了图表类型Worldmap，可帮助你制作呈现各国数据的世界地图。为演示如何使用Worldmap，我们来创建一个突出北美、中美和南美的简单地图：

americas.py

```
import pygal

❶ wm = pygal.Worldmap()
   wm.title = 'North, Central, and South America'

❷ wm.add('North America', ['ca', 'mx', 'us'])
   wm.add('Central America', ['bz', 'cr', 'gt', 'hn', 'ni', 'pa', 'sv'])
   wm.add('South America', ['ar', 'bo', 'br', 'cl', 'co', 'ec', 'gf',
                             'gy', 'pe', 'py', 'sr', 'uy', 've'])

❸ wm.render_to_file('americas.svg')
```

在❶处，我们创建了一个Worldmap实例，并设置了该地图的title属性。在❷处，我们使用了方法add()，它接受一个标签和一个列表，其中后者包含我们要突出的国家的国别码。每次调用add()都将为指定的国家选择一种新颜色，并在图表左边显示该颜色和指定的标签。我们要以同一种颜色显示整个北美地区，因此第一次调用add()时，在传递给它的列表中包含'ca'、'mx'和'us'，以同时突出加拿大、墨西哥和美国。接下来，对中美和南美国家做同样的处理。

❸处的方法render_to_file()创建一个包含该图表的.svg文件，你可以在浏览器中打开它。输出一幅以不同颜色突出北美、中美和南美的地图，如图16-7所示。



图16-7 图表类型Worldmap的一个简单实例

知道如何创建包含彩色区域、颜色标示和标签的地图后，我们在地图中添加数据，以显示有关国家的信息。

16.2.6 在世界地图上呈现数字数据

为练习在地图上呈现数字数据，我们来创建一幅地图，显示三个北美国家的人口数量：

na_populations.py

```
import pygal

wm = pygal.Worldmap()
wm.title = 'Populations of Countries in North America'
❶ wm.add('North America', {'ca': 34126000, 'us': 309349000, 'mx': 113423000})

wm.render_to_file('na_populations.svg')
```

首先，创建了一个Worldmap实例并设置了标题。接下来，使用了方法add()，但这次通过第二个实参传递了一个字典而不是列表（见❶）。这个字典将两个字母的Pygal国别码作为键，将人口数量作为值。Pygal根据这些数字自动给不同国家着以深浅不一的颜色（人口最少的国家颜色最浅，人口最多的国家颜色最深），如图16-8所示。



图16-8 北美国家的人口数量

这幅地图具有交互性：如果你将鼠标指向某个国家，将看到其人口数量。下面在这个地图中添加更多的数据。

16.2.7 绘制完整的世界人口地图

要呈现其他国家的人口数量，需要将前面处理的数据转换为Pygal要求的字典格式：键为两个字母的国别码，值为人口数量。为此，在world_population.py中添加如下代码：

world_population.py

```
import json

import pygal

from country_codes import get_country_code

# 将数据加载到列表中
--snip--

# 创建一个包含人口数量的字典
❶ cc_populations = {}
for pop_dict in pop_data:
    if pop_dict['Year'] == '2010':
        country = pop_dict['Country Name']
        population = int(float(pop_dict['Value']))
```



```

code = get_country_code(country)
if code:
    ❷ cc_populations[code] = population

    ❸ wm = pygal.Worldmap()
    wm.title = 'World Population in 2010, by Country'
    ❹ wm.add('2010', cc_populations)

wm.render_to_file('world_population.svg')

```

我们首先导入了pygal。在❶处，我们创建了一个空字典，用于以Pygal要求的格式存储国别码和人口数量。在❷处，如果返回了国别码，就将国别码和人口数量分别作为键和值填充字典cc_populations。我们还删除了所有的print语句。

在❸处，我们创建了一个Worldmap实例，并设置其title属性。在❹处，我们调用了add()，并向它传递由国别码和人口数量组成的字典。图16-9显示了生成的地图。



图16-9 2010年的世界人口数量

有几个国家没有相关的数据，我们将其显示为黑色，但对于大多数国家，都根据其人口数量进行了着色。本章后面将处理数据缺失的问题，这里先来修改着色，以更准确地反映各国的人口数量。在当前的地图中，很多国家都是浅色的，只有两个国家是深色的。对大多数国家而言，颜色深浅的差别不足以反映其人口数量的差别。为修复这种问题，我们将根据人口数量将国家分组，再分别给每个组着色。

16.2.8 根据人口数量将国家分组

印度和中国的人口比其他国家多得多，但在当前的地图中，它们的颜色与其他国家差别较小。中国和印度的人口都超过了10亿，接下来人口最多的国家是美国，但只有大约3亿。下面不将所有国家都作为一个编组，而是根据人口数量分成三组——少于1000万的、介于1000万和10亿之间的以及超过10亿的：

world_population.py

```
--snip--
# 创建一个包含人口数据的字典
cc_populations = {}
for pop_dict in pop_data:
    if pop_dict['Year'] == '2010':
        --snip--
        if code:
            cc_populations[code] = population

# 根据人口数量将所有的国家分成三组
❶ cc_pops_1, cc_pops_2, cc_pops_3 = {}, {}, {}
❷ for cc, pop in cc_populations.items():
    if pop < 10000000:
        cc_pops_1[cc] = pop
    elif pop < 1000000000:
        cc_pops_2[cc] = pop
    else:
        cc_pops_3[cc] = pop

# 看看每组分别包含多少个国家
❸ print(len(cc_pops_1), len(cc_pops_2), len(cc_pops_3))

wm = pygal.Worldmap()
wm.title = 'World Population in 2010, by Country'
❹ wm.add('0-10m', cc_pops_1)
wm.add('10m-1bn', cc_pops_2)
wm.add('>1bn', cc_pops_3)

wm.render_to_file('world_population.svg')
```

为将国家分组，我们创建了三个空字典（见❶）。接下来，遍历cc_populations，检查每个国家的人口数量（见❷）。if-elif-else代码块将每个国别码-人口数量对加入到合适的字典（cc_pops_1、cc_pops_2或cc_pops_3）中。

在❸处，我们打印这些字典的长度，以获悉每个分组的规模。绘制地图时，我们将全部三个分组都添加到Worldmap中（见❹）。如果你现在运行这个程序，首先看到的将是每个分组的规模：

```
85 69 2
```

上述输出表明，人口少于1000万的国家有85个，人口介于1000万和10亿之间的国家有69个，

还有两个国家比较特殊，其人口都超过了10亿。这样的分组看起来足够了，让地图包含丰富的信息。图16-10显示了生成的地图。



图16-10 分三组显示的世界各国人口

现在使用了三种不同的颜色，让我们能够看出人口数量上的差别。在每组中，各个国家都按人口从少到多着以从浅到深的颜色。

16.2.9 使用 Pygal 设置世界地图的样式

在这个地图中，根据人口将国家分组虽然很有效，但默认的颜色设置很难看。例如，在这里，Pygal选择了鲜艳的粉色和绿色基色。下面使用Pygal样式设置指令来调整颜色。

我们也让Pygal使用一种基色，并将指定该基色，并让三个分组的颜色差别更大：

world_population.py

```
import json

import pygal
❶ from pygal.style import RotateStyle
--snip--
# 根据人口数量将所有的国家分成三组
cc_pops_1, cc_pops_2, cc_pops_3 = {}, {}, {}
for cc, pop in cc_populations.items():
    if pop < 100000000:
        --snip--
```

```
❷ wm_style = RotateStyle('#336699')
❸ wm = pygal.Worldmap(style=wm_style)
  wm.title = 'World Population in 2010, by Country'
  --snip--
```

Pygal样式存储在模块style中，我们从这个模块中导入了样式RotateStyle（见❶）。创建这个类的实例时，需要提供一个实参——十六进制的RGB颜色（见❷）；Pygal将根据指定的颜色为每组选择颜色。十六进制格式的RGB颜色是一个以井号（#）打头的字符串，后面跟着6个字符，其中前两个字符表示红色分量，接下来的两个表示绿色分量，最后两个表示蓝色分量。每个分量的取值范围为00（没有相应的颜色）~FF（包含最多的相应颜色）。如果你在线搜索hex color chooser（十六进制颜色选择器），可找到让你能够尝试选择不同的颜色并显示其RGB值的工具。这里使用的颜色值（#336699）混合了少量的红色（33）、多一些的绿色（66）和更多一些的蓝色（99），它为RotateStyle提供了一种淡蓝色基色。

RotateStyle返回一个样式对象，我们将其存储在wm_style中。为使用这个样式对象，我们在创建Worldmap实例时以关键字实参的方式传递它（见❸）。更新后的地图如图16-11所示。



图16-11 按人口划分的三个国家编组了统一的颜色主题

前面的样式设置让地图的颜色更一致，也更容易区分不同的编组。

16.2.10 加亮颜色主题

Pygal通常默认使用较暗的颜色主题。为方便印刷，我使用LightColorizedStyle加亮了地图

的颜色。这个类修改整个图表的主题，包括背景色、标签以及各个国家的颜色。要使用这个样式，先导入它：

```
from pygal.style import LightColorizedStyle
```

然后就可独立地使用LightColorizedStyle了，例如：

```
wm_style = LightColorizedStyle
```

然而使用这个类时，你不能直接控制使用的颜色，Pygal将选择默认的基色。要设置颜色，可使用RotateStyle，并将LightColorizedStyle作为基本样式。为此，导入LightColorizedStyle和RotateStyle：

```
from pygal.style import LightColorizedStyle, RotateStyle
```

再使用RotateStyle创建一种样式，并传入另一个实参base_style：

```
wm_style = RotateStyle('#336699', base_style=LightColorizedStyle)
```

这设置了较亮的主题，同时根据通过实参传递的颜色给各个国家着色。使用这种样式时，生成的图表与本书的屏幕截图更一致。

尝试为不同的可视化选择合适的样式设置指令时，在import语句中指定别名会有所帮助：

```
from pygal.style import LightColorizedStyle as LCS, RotateStyle as RS
```

这样，样式定义将更短：

```
wm_style = RS('#336699', base_style=LCS)
```

通过使用几个样式设置指令，就能很好地控制图表和地图的外观。

动手试一试

16-5 涵盖所有国家：本节制作人口地图时，对于大约 12 个国家，程序不能自动确定其两个字母的国别码。请找出这些国家，在字典 COUNTRIES 中找到它们的国别码；然后，对于每个这样的国家，都在 get_country_code() 中添加一个 if-elif 代码块，以返回其国别码：

```
if country_name == 'Yemen, Rep.':
    return 'ye'
elif --snip--
```

将这些代码放在遍历 COUNTRIES 的循环和语句 return None 之间。完成这样的修改

后，你看到的地图将更完整。

16-6 国内生产总值：Open Knowledge Foundation 提供了一个数据集，其中包含全球各国的国内生产总值（GDP），可在 <http://data.okfn.org/data/core/gdp/> 找到这个数据集。请下载这个数据集的 JSON 版本，并绘制一个图表，将全球各国最近一年的 GDP 呈现出来。

16-7 选择你自己的数据：世界银行（The World Bank）提供了很多数据集，其中包含有关全球各国的信息。请访问 <http://data.worldbank.org/indicator/>，并找到一个你感兴趣的数据集。单击该数据集，再单击链接 Download Data 并选择 CSV。你将收到三个 CSV 文件，其中两个包含字样 Metadata，你应使用第三个 CSV 文件。编写一个程序，生成一个字典，它将两个字母的 Pygal 国别码作为键，并将你从这个文件中选择的数据作为值。使用 Worldmap 制作一个地图，在其中呈现这些数据，并根据你的喜好设置这个地图的样式。

16-8 测试模块 country_codes：我们编写模块 country_codes 时，使用了 print 语句来核实 get_country_code() 能否按预期那样工作。请利用你在第 11 章学到的知识，为这个函数编写合适的测试。

16.3 小结

在本章中，你学习了：如何使用网上的数据集；如何处理 CSV 和 JSON 文件，以及如何提取你感兴趣的数据；如何使用 matplotlib 来处理以往的天气数据，包括如何使用模块 datetime，以及如何在同一个图表中绘制多个数据系列；如何使用 Pygal 绘制呈现各国数据的世界地图，以及如何设置 Pygal 地图和图表的样式。

有了使用 CSV 和 JSON 文件的经验后，你将能够处理几乎任何要分析的数据。大多数在线数据集都可以以这两种格式中的一种或两种下载。学习使用这两种格式为学习使用其他格式的数据做好了准备。

在下一章，你将编写自动从网上采集数据并对其进行可视化的程序。如果你只是将编程作为业余爱好，学会这些技能可以增加乐趣；如果你有志于成为专业程序员，就必须掌握这些技能。